
Rekayasa Sistem Cerdas

Arsitektur dan Representasi Pengetahuan

First edition

Armein Z. R. Langi

Rekayasa Sistem Cerdas

Arsitektur dan Representasi Pengetahuan

First edition

This version of can be viewed and downloaded free of charge for personal use only. It must not be redistributed, sold, or used in derivative works.

© Armein Z. R. Langi, 2026.

Table of contents

Pengantar	7
Bagian 1: Konsep Inti Ontologi	9
1. Sistem Cerdas & Siklus PUDAL	11
1.1. Perceive (Persepsi)	12
1.2. Understand (Memahami)	13
1.3. Decision (Keputusan)	14
1.4. Act (Tindakan)	15
1.5. Learning (Belajar)	16
2. Artefak PSKVE	19
2.1. Product (Produk)	20
2.2. Service (Layanan)	22
2.3. Knowledge (Pengetahuan)	23
2.4. Value (Nilai)	24
2.5. Environment (Lingkungan)	26
3. Energi Multi-Dimensi	29
3.1. Energi Produk (Fisik)	30
3.2. Energi Layanan (Atensi)	32
3.3. Energi Pengetahuan (Algoritma)	33
3.4. Energi Nilai (Finansial/Sosial)	35
3.5. Energi Ruang Lingkungan (Inspirasi)	36
3.6. Konversi Transaksional	38
4. Smart Engine Abstraction (SEA)	41
Bagian 2: Metodologi Multi-Domain (PICOC)	43
5. Domain Aplikasi (A)	45
5.1. P: Stakeholder	46
5.2. I: Solusi Baru	47
5.3. Cx: Masalah Spesifik	48
6. Domain Sistem (S)	51
6.1. P: Testbeds/Dataset	52
6.2. I: Arsitektur Sistem Cerdas	54
7. Domain Teknologi (T)	57
7.1. P: Energi Sumber/Data Mentah	59

Table of contents

7.2. I: Engine/Modul Teknologi	60
8. Domain Riset Fundamental (F)	63
8.1. P: Fenomena/Prinsip Sains	64
8.2. I: Teori/Model Baru	66
Bagian 3: Integrasi Teknologi	69
9. Prolog (Penalaran Simbolik)	71
9.1. Representasi Fakta & Aturan	72
9.2. Mesin Inferensi Logika	74
9.3. Pemetaan Komponen Ontologi	75
10. Python (Algoritma & Aplikasi)	79
10.1. Pemrosesan Data	80
10.2. Antarmuka Pengguna	82
10.3. Orkestrasi Simulasi	83
11. Bridge Libraries	85
11.1. PySWIP	86
11.2. Janus (SWI-Prolog)	88
11.3. PrologMQI	90
Bagian 4: Studi Kasus	91
12. Studi Kasus: Transportasi 2030	93
12.1. Skenario Bandung-Jakarta	95
12.2. Evaluasi Fleet EV vs Bus vs Kereta Cepat	96
12.3. Metrik: Waktu, Biaya, Emisi CO2	98
Daftar Pustaka	101

List of figures

List of tables

Pengantar

Buku ini memperkenalkan kerangka kerja **Smart Engineering** yang mengintegrasikan kecerdasan buatan dan alami melalui siklus **PUDAL** untuk menciptakan artefak **PSKVE**. Penulis mengusulkan penggunaan **ontologi** sebagai spesifikasi konsep yang menghubungkan domain aplikasi, sistem, teknologi, dan penelitian fundamental dalam satu struktur terpadu. Implementasi teknisnya memanfaatkan kekuatan gabungan antara **Prolog** untuk penalaran logika simbolik dan **Python** untuk pengolahan data serta simulasi interaktif. Kerangka kerja ini diterapkan dalam studi kasus transportasi masa depan untuk mengevaluasi efisiensi layanan, biaya, dan dampak lingkungan secara holistik. Melalui konsep **Smart Engine Abstraction (SEA)**, metode ini menjembatani model konseptual manusia dengan struktur data mesin guna menyelesaikan tantangan rekayasa multidisiplin yang kompleks. Keseluruhan teks memberikan panduan metodologis bagi insinyur untuk membangun sistem cerdas yang mampu melakukan transformasi energi fisik maupun nilai informasi secara transaksional.

Bagian 1: Konsep Inti Ontologi

Secara formal, **ontologi** didefinisikan sebagai spesifikasi eksplisit dari konseptualisasi bersama yang memodelkan suatu domain pengetahuan. Ontologi menyediakan primitif representasional yang terstruktur sehingga data dan relasi di dalamnya dapat dipahami, dikueri, dan dinalar oleh mesin.

Konsep Inti Ontologi Dalam ilmu komputer dan informasi, komponen bangunan fundamental dari ontologi terdiri atas empat elemen:

- **Kelas (*Classes/Concepts*):** Pengelompokan abstrak yang mewakili kumpulan objek dengan karakteristik yang sama (misalnya: entitas Mahasiswa, Kursus, atau Kendaraan). Kelas dapat disusun secara hierarkis sehingga kelas yang lebih spesifik dapat mewarisi properti dari kelas yang lebih umum.
- **Individu (*Individuals/Instances*):** Entitas dasar, konkret, atau spesifik yang menjadi anggota dari suatu kelas. Sebagai contoh, ‘Alice Wonderland’ adalah individu dari kelas Mahasiswa, atau ‘EV_Sedan’ adalah individu dari kelas Kendaraan.
- **Properti (*Properties/Attributes*):** Karakteristik spesifik yang mendeskripsikan individu dalam kelas tertentu, lengkap dengan nama properti dan tipe nilainya (misalnya: ID mahasiswa, atau nilai kecepatan sebuah kendaraan).
- **Relasi (*Relationships*):** Keterkaitan makna semantik yang menghubungkan kelas maupun individu di dalam ontologi. Relasi ini membentuk pola jaringan kompleks, seperti hierarki kepemilikan (*is_a*), komposisi (*part_of*), atau interaksi aksi (misalnya, mahasiswa *enrolls_in* sebuah kursus).

Konteksnya dalam Smart Engineering Framework Dalam paradigma *Smart Engineering*, konsep inti ontologi di atas dimanfaatkan secara lebih luas sebagai fondasi model konseptual untuk merancang, memahami, dan memecahkan permasalahan teknis kompleks lintas domain. Peran ontologi dalam kerangka kerja ini mencakup:

- **Pondasi untuk Siklus Kecerdasan (PUDAL):** *Smart Engineering* berpusat pada integrasi Sistem Cerdas (sistem alami dan buatan) yang terus-menerus beroperasi melalui siklus PUDAL: *Perceive, Understand, Decision-making & planning, Act-Response, Learning-evaluating*. Ontologi

memberikan basis pengetahuan formal bagi mesin pada tahap “**Understand**” dan “**Decision-making**”, yang memungkinkan agen atau kendaraan cerdas menginterpretasikan keadaan lingkungan serta menarik kesimpulan strategis.

- **Pemodelan Energi PSKVE Multidimensi:** Kerangka ontologi *Smart Engineering* memperluas konsep energi yang biasanya murni berupa fisik menjadi wujud nilai yang lebih luas, yaitu **PSKVE** (*Product, Service, Knowledge, Value, Environment*). Ontologi menstrukturkan bagaimana entitas-entitas teknis mampu melakukan “Konversi Transaksional” antardimensi ini. Misalnya, mengonversi pengetahuan (seperti algoritma baterai EV) menjadi energi nilai (pengurangan biaya) atau layanan (waktu tempuh).
- **Pemecahan Masalah 4 Domain (PICOC):** Ontologi memungkinkan pemecahan sebuah inovasi teknik (seperti desain transportasi canggih) ke dalam empat lapisan domain: *Application* (Kebutuhan Pengguna), *System* (Kendaraan/Sistem Utama), *Technology* (Mesin/Teknologi Inti), dan *Fundamental Research* (Prinsip Dasar).
- **Implementasi Platform Hibrida (Prolog-Python):** Untuk mengimplementasikan ontologi ke dalam aplikasi teknik yang nyata, kerangka *Smart Engineering* menggunakan bahasa **Prolog** (di mana kelas, properti, individu, dan relasi ontologi dikonversi menjadi fakta dan aturan logika) yang digabungkan dengan **Python** sebagai penyedia daya algoritmik dan antarmuka simulasi. Sebagai contoh praktis, ontologi Prolog digunakan untuk menyimpan data properti kendaraan dan rute untuk tahun 2030, lalu aplikasi Python menjalankan simulasi kueri untuk mengalkulasi efisiensi biaya maupun pengurangan emisi CO2 lintas rute Bandung-Jakarta.

1

Sistem Cerdas & Siklus PUDAL

Dalam kerangka *Smart Engineering*, **Sistem Cerdas** (*Intelligent Systems*) didefinisikan sebagai entitas yang mampu menjalankan siklus **PUDAL** (*Perceive, Understand, Decision-making & planning, Act-Response, Learning-evaluating*). Siklus fundamental ini membuat sebuah sistem mampu menerima stimulus eksternal, bersifat adaptif, memiliki otonomi, berorientasi pada tujuan, serta mampu menghasilkan respons dan peningkatan performa internal.

Ontologi berperan sebagai lapisan fondasi yang sangat penting bagi arsitektur Sistem Cerdas karena menyediakan kerangka kosakata dan struktur data yang dapat dimengerti oleh mesin (*machine-understandable*). Jika dihubungkan dengan **Konsep Inti Ontologi** (yang berpusat pada elemen *Kelas, Individu, Properti*, dan *Relasi*), tahapan-tahapan siklus PUDAL pada Sistem Cerdas beroperasi sebagai berikut:

- **Perceive (Mempersepsikan) & Understand (Memahami):** Ketika Sistem Cerdas mengumpulkan data mentah (stimulus) dari lingkungannya melalui sensor, ontologi digunakan untuk menyusun dan menginterpretasikan data tersebut menjadi model representasi realitas. Data ini diklasifikasikan sebagai representasi **Individu** (entitas spesifik) yang merupakan bagian dari **Kelas** (kategori konsep) tertentu. Mesin memberikan makna pada data dengan memetakan **Properti** (atribut data) dan membangun **Relasi** (keterkaitan semantik) dari objek-objek tersebut sesuai kerangka ontologi yang disepakati.
- **Decision-making & Planning (Pengambilan Keputusan):** Berdasarkan pemahaman terstruktur dari langkah sebelumnya, sistem

1. Sistem Cerdas & Siklus PUDAL

memilih tindakan atau rencana yang paling tepat. Basis pengetahuan ontologis memberdayakan sistem mesin logika (seperti bahasa Prolog) untuk melakukan penalaran algoritmik otomatis (*automated reasoning*). Sistem menggunakan aturan-aturan logis (*rules*) terhadap relasi dan kelas yang ada untuk menarik kesimpulan logis baru yang belum tertulis secara eksplisit, yang sangat krusial dalam pengambilan keputusan.

- **Act-Response (Bertindak) & Learning-evaluating (Mengevaluasi/Belajar):** Sistem kemudian mengeksekusi pilihan aksinya ke dalam modifikasi lingkungan dan menilai apakah hasil aksinya berhasil. Proses “belajar” pada mesin dapat diwujudkan dengan menggunakan ontologi dinamis, di mana hasil dari evaluasi aksi dapat dikonversi menjadi fakta logis baru. Sistem memperbarui basis pengetahuannya dengan memodifikasi **Properti**, memasukkan **Individu** baru, atau mendefinisikan **Relasi** baru, yang akan meningkatkan performa pada siklus PUDAL di masa depan.

Dalam konteks *Smart Engineering* yang lebih luas, ontologi dan siklus PUDAL diaplikasikan dalam dua bentuk utama:

1. **Pada Artefak Rekayasa (PSVKE):** Produk akhir yang dihasilkan berupa artefak bernilai PSVKE (*Product, Service, Knowledge, Value, Environment*) yang tersemat komponen cerdas di dalamnya, di mana mesin berinteraksi secara cerdas dengan penggunaanya.
2. **Pada Proses Rekayasanya Sendiri:** Proses desain dan evaluasi dari tim insinyur itu sendiri diperlakukan layaknya sebuah sistem cerdas yang menjalankan siklus mirip PUDAL. Tim menggunakan pemodelan ontologi untuk mengurai kompleksitas secara interdisipliner, memastikan model konseptual dari pikiran manusia selaras dengan struktur data logis yang akan dinalar oleh sistem komputasi (kecerdasan buatan). Paradigma gabungan (*human-machine paradigm*) ini dirancang untuk dapat mengamplifikasi kemampuan dari kecerdasan alami manusia melalui komputasi.

1.1. Perceive (Persepsi)

Dalam siklus PUDAL (*Perceive, Understand, Decision-making & planning, Act-Response, Learning-evaluating*) yang menjadi karakteristik utama dari sebuah Sistem Cerdas, **tahap Perceive (Persepsi) berfungsi sebagai langkah paling awal dan gerbang masuknya informasi**. Siklus ini selalu diawali atau dipicu ketika sebuah sistem cerdas menerima stimulus.

Secara spesifik, **tahap Perceive adalah proses di mana sistem mengumpulkan data, baik dari lingkungan eksternalnya maupun dari status internalnya sendiri**. Untuk menangkap data tersebut, sistem cerdas

menggunakan sensor atau berbagai mekanisme masukan (input) lainnya. Data mentah yang berhasil dikumpulkan pada tahap persepsi ini sangat esensial karena menjadi bahan dasar yang akan diteruskan ke tahap *Understand*, di mana sistem memproses dan menginterpretasikan data tersebut untuk membangun model pemahaman terhadap situasi saat itu.

Dalam paradigma *Smart Engineering*, konsep “Perceive” diterapkan secara berlapis pada dua jenis Sistem Cerdas:

- **Pada Artefak atau Sistem Mesin:** Entitas cerdas (seperti kendaraan otonom atau platform digital) mengandalkan sensor fisikal maupun input data untuk mempersepsikan lingkungannya. Kemampuan persepsi inilah yang pada akhirnya memungkinkan mesin untuk menjadi entitas yang adaptif, berorientasi pada tujuan, dan dapat bertindak secara otonom di dunia nyata.
- **Pada Proses Rekayasa (Tim Insinyur):** Proses perancangan (*engineering process*) yang dilakukan oleh manusia juga dipandang sebagai sebuah sistem cerdas yang menjalankan siklus menyerupai PUDAL. Dalam konteks ini, tahap “Perceive” terjadi ketika **para insinyur mempersepsikan atau menangkap kebutuhan pengguna (user needs) beserta batasan-batasan (constraints) dari masalah yang ada**. Persepsi awal dari insinyur ini menjadi landasan untuk memahami domain masalah sebelum merancang dan mengimplementasikan artefak.

Singkatnya, tahapan *Perceive* merupakan fondasi observasional yang krusial, tanpanya Sistem Cerdas (baik yang berupa kecerdasan buatan maupun tim perancang manusia) tidak akan memiliki konteks apa pun untuk dipahami, diputuskan, dan dipelajari.

1.2. Understand (Memahami)

Dalam siklus PUDAL (*Perceive, Understand, Decision-making & planning, Act-Response, Learning-evaluating*) yang menjadi motor penggerak sebuah Sistem Cerdas, tahap **Understand (Memahami) merupakan proses di mana sistem memproses dan menginterpretasikan data yang telah dikumpulkan pada tahap persepsi**.

Tujuan utama dari tahapan ini adalah **untuk membangun sebuah model atau representasi dari situasi saat itu beserta konteksnya**. Jika tahap *Perceive* (Persepsi) hanya berfungsi menangkap data mentah (stimulus) dari lingkungan, maka tahap *Understand* adalah momen di mana sistem—baik itu mesin buatan maupun pikiran manusia—memberikan makna, struktur, dan konteks pada data tersebut.

1. Sistem Cerdas & Siklus PUDAL

Dalam paradigma *Smart Engineering*, proses “Memahami” ini diaplikasikan pada dua dimensi utama:

- **Pada Artefak atau Sistem Komputasi:** Agar sebuah mesin buatan dapat “memahami” lingkungannya, ia memerlukan struktur pengetahuan formal, seperti **ontologi**, yang membuat data tersebut dapat dimengerti oleh mesin (*machine-understandable*). Pada tahap ini, agen cerdas mengonversi sinyal sensor menjadi representasi konseptual (misalnya, mengenali bahwa suatu objek adalah entitas dari kelas tertentu dengan properti khusus). Representasi situasional inilah yang mutlak diperlukan sebelum mesin dapat beralih ke tahap *Decision-making & Planning* untuk memilih tindakan yang rasional dan adaptif.
- **Pada Proses Rekayasa (Tim Insinyur):** Desain rekayasa itu sendiri dipandang sebagai sebuah sistem cerdas di mana tim insinyur manusia juga beroperasi melalui siklus yang mirip dengan PUDAL. Dalam konteks manusia, tahap *Understand* direpresentasikan oleh momen di mana **insinyur memahami domain permasalahan secara holistik setelah mempersepsikan kebutuhan pengguna dan batasan-batasan desain**. Pemahaman konseptual yang mendalam ini sangat krusial agar solusi teknis yang akan dirancang benar-benar relevan dengan masalah yang ada.

Secara keseluruhan, tahapan *Understand* bertindak sebagai **jembatan kognitif** yang vital. Tanpa kemampuan untuk menerjemahkan data mentah menjadi sebuah representasi dan konteks yang terstruktur, suatu Sistem Cerdas tidak akan memiliki dasar pengetahuan (baik logis maupun intuitif) untuk merencanakan keputusan atau belajar dari pengalaman.

1.3. Decision (Keputusan)

Dalam siklus PUDAL (*Perceive, Understand, Decision-making & planning, Act-Response, Learning-evaluating*) yang menjadi inti penggerak sebuah Sistem Cerdas, tahap **Decision-making & Planning (Pengambilan Keputusan dan Perencanaan)** merupakan fase penentuan arah tindakan. Pada tahap ini, **sistem secara spesifik memilih rencana atau tindakan yang paling sesuai berdasarkan pemahaman situasional dan tujuan yang ingin dicapainya**.

Fase pengambilan keputusan ini sangat krusial karena ia secara fundamental memungkinkan sebuah Sistem Cerdas untuk menjadi entitas yang adaptif, berorientasi pada tujuan, serta memiliki kemampuan untuk beroperasi secara otonom dalam berbagai tingkatan.

Dalam kerangka kerja *Smart Engineering*, proses “Keputusan” ini diterapkan pada dua area fungsional:

- **Pengambilan Keputusan oleh Komputasi Mesin:** Agar mesin dapat mengambil keputusan yang rasional dan cerdas, mereka mengandalkan kemampuan penalaran dari platform berbasis pengetahuan (seperti integrasi bahasa pemrograman Prolog). Pada tahap ini, mesin memproses data yang telah terstruktur menggunakan aturan logika formal dan mesin inferensi untuk **mengevaluasi batasan, memverifikasi konsistensi, dan menarik kesimpulan logis yang tidak terlihat secara kasat mata (non-obvious conclusions)**. Contoh nyata dari tahap ini pada sistem mesin adalah saat program secara otomatis menghasilkan keputusan diagnostik dari suatu kegagalan alat atau mengevaluasi apakah suatu syarat desain sudah terpenuhi sebelum melakukan eksekusi sistem.
- **Pengambilan Keputusan oleh Tim Insinyur Manusia:** Paradigma *Smart Engineering* juga memandang proses desain yang dilakukan oleh manusia sebagai sebuah Sistem Cerdas itu sendiri. Dalam proses operasional tim insinyur, setelah mereka selesai menangkap batasan (Persepsi) dan menganalisis domain permasalahan (Memahami), tahap keputusan direpresentasikan sebagai momen di mana **para insinyur secara kolektif membuat keputusan-keputusan desain teknis**. Keputusan desain inilah yang pada akhirnya menjadi dasar rencana yang akan diwujudkan dalam implementasi pembuatan purwarupa (yang masuk ke dalam fase *Act*).

Secara keseluruhan, tahap Pengambilan Keputusan merupakan titik transisi utama di mana sebuah Sistem Cerdas mengonversi pemahaman observasionalnya menjadi sebuah rencana intervensi strategis untuk menyelesaikan masalah atau mencapai target yang ditetapkan.

1.4. Act (Tindakan)

Dalam siklus PUDAL (*Perceive, Understand, Decision-making & planning, Act-Response, Learning-evaluating*) yang mendasari Sistem Cerdas, tahap **Act-Response (Tindakan)** adalah fase eksekusi di mana rencana atau keputusan diwujudkan ke dalam bentuk tindakan nyata.

Setelah sistem memproses data (Persepsi), membangun konteks (Memahami), dan memilih strategi (Keputusan), tahap *Act-Response* merupakan **momen di mana sistem secara aktif mengeksekusi tindakan yang dipilihnya untuk berinteraksi dengan, atau memodifikasi, lingkungannya**. Tahap ini merepresentasikan “respons” langsung sistem terhadap “stimulus” awal yang diterimanya.

1. Sistem Cerdas & Siklus PUDAL

Sama seperti tahapan sebelumnya dalam kerangka *Smart Engineering*, konsep “Tindakan” ini diimplementasikan dalam dua dimensi utama:

- **Pada Sistem Mesin atau Artefak Cerdas:** Tahap *Act* terjadi ketika mesin melakukan intervensi fisik atau komputasional. Berdasarkan konsep *Smart Engine Abstraction* (SEA), entitas mesin cerdas bertindak dengan **mengonversi input (seperti gaya atau energi mentah) menjadi output yang memiliki nilai (solusi atau pekerjaan)**. Sebagai contoh pada sistem transportasi, tahap ini adalah saat sistem kontrol kendaraan (*vehicle*) benar-benar mengeksekusi navigasi fisik di jalan raya setelah memutuskan rute terbaik.
- **Pada Proses Rekayasa (Tim Insinyur Manusia):** Dalam memandang proses kerja insinyur sebagai sebuah sistem cerdas, tahap *Act* merupakan transisi dari sekadar konsep di atas kertas menjadi realitas fisik. Para insinyur “bertindak” melalui **kegiatan implementasi teknis dan pembuatan purwarupa (prototyping)**. Mereka mengeksekusi desain yang telah diputuskan sebelumnya untuk menciptakan artefak rekayasa yang nyata.

Dalam konteks siklus PUDAL secara keseluruhan, tahap **Tindakan (Act)** **bertindak sebagai pemicu perubahan di dunia nyata**. Eksekusi dari mesin maupun insinyur ini akan menghasilkan dampak atau memodifikasi lingkungan sekitarnya. Efek dari tindakan inilah yang kemudian akan diukur, dinilai, dan dijadikan bahan pembelajaran pada tahap akhir siklus, yaitu *Learning-evaluating* (Belajar/Mengevaluasi).

1.5. Learning (Belajar)

Dalam siklus PUDAL (*Perceive, Understand, Decision-making & planning, Act-Response, Learning-evaluating*), tahap **Learning-evaluating (Belajar dan Mengevaluasi)** adalah fase pamungkas yang memastikan sebuah Sistem Cerdas dapat terus berkembang.

Setelah sistem mengeksekusi sebuah keputusan di dunia nyata pada tahap *Act*, tahap *Learning* adalah proses di mana **sistem menilai atau mengevaluasi hasil (kegagalan maupun keberhasilan) dari tindakan tersebut**. Berdasarkan hasil penilaian ini, sistem kemudian **memperbarui pengetahuan atau model internalnya untuk meningkatkan performanya di masa depan**. Fase ini berfokus pada upaya sistem untuk menghasilkan perbaikan internal.

Kemampuan “Belajar” ini merupakan hal yang sangat fundamental karena inilah **kunci utama yang membuat sebuah sistem mampu menjadi**

entitas yang adaptif, memiliki otonomi, dan secara konsisten berorientasi pada tujuan.

Dalam kerangka *Smart Engineering*, konsep belajar dan mengevaluasi ini diterapkan secara berkesinambungan pada dua elemen utama:

- **Pada Sistem Mesin (Artefak Cerdas):** Setelah agen cerdas (seperti kendaraan otonom atau perangkat lunak analitik) bertindak, ia mengukur efek dari tindakannya tersebut. Mesin kemudian memodifikasi basis pengetahuannya (seperti memperbarui fakta atau aturan dalam model ontologi/Prolog) agar perhitungan dan keputusannya di siklus PUDAL berikutnya menjadi lebih akurat, efisien, dan relevan dengan perubahan lingkungan.
- **Pada Proses Rekayasa (Tim Insinyur):** Proses desain rekayasa yang dilakukan manusia juga mengandalkan tahap evaluasi ini. Bagi para insinyur, fase *Learning* terjadi ketika mereka **mengevaluasi hasil pengujian (testing) dan mengumpulkan umpan balik (feedback) dari purwarupa yang telah mereka buat**. Evaluasi observasional ini memberi para insinyur pengetahuan baru yang digunakan untuk melakukan iterasi, merevisi desain, dan memperbaiki solusi teknis pada siklus pengembangan selanjutnya.

2

Artefak PSKVE

Dalam paradigma *Smart Engineering*, hasil akhir dari proses rekayasa tidak lagi dipandang sekadar sebagai benda fisik mekanis, melainkan sebagai **Artefak PSKVE** (*Product, Service, Knowledge, Value, Environment*). Artefak ini dirancang untuk menjadi entitas cerdas—atau memiliki komponen kecerdasan di dalamnya—yang mampu berinteraksi secara holistik dengan pengguna maupun lingkungannya.

Agar mesin komputasi atau perangkat lunak dapat merancang, mengevaluasi, dan menyimulasikan kompleksitas dari artefak cerdas ini, kerangka kerjanya menggunakan **Konsep Inti Ontologi**. Ontologi menyediakan struktur formal bagi sistem (seperti basis pengetahuan pada bahasa Prolog) untuk memetakan kelima dimensi PSKVE ke dalam bentuk yang dapat dinalar oleh mesin (*machine-understandable*), menggunakan empat komponen bangunan utamanya: Kelas, Individu, Properti, dan Relasi.

Hubungan antara Artefak PSKVE dengan Konsep Inti Ontologi dapat dipetakan sebagai berikut:

- **Kelas (Classes) dan Individu (Individuals) sebagai Wujud Artefak:**
Di dalam basis pengetahuan ontologis, entitas fisik atau abstrak dari Artefak PSKVE didefinisikan sebagai **Individu** yang bernaung di bawah suatu **Kelas**. Sebagai contoh nyata dalam simulasi transportasi cerdas, “EV_Sedan” atau “High-Speed Electric Train” adalah entitas individu spesifik di dalam kelas “Kendaraan” (merepresentasikan dimensi *Product* secara fisik) yang keberadaannya dideklarasikan secara eksplisit agar mesin dapat mengenalinya.

2. Artefak PSKVE

- **Properti (Properties) sebagai Pengukur Dimensi PSKVE:** Kelima dimensi energi PSKVE yang abstrak tersebut diterjemahkan secara komputasional menjadi atribut atau **Properti** yang melekat pada individu artefak tersebut.
- **Product Energy** (energi fisik artefak) dimodelkan melalui properti teknis seperti kapasitas penumpang atau konsumsi bahan bakar.
- **Service Energy** (interaksi dan layanan) dimodelkan melalui properti performa seperti estimasi waktu tempuh atau tingkat keandalan (MTBF).
- **Knowledge Energy** (algoritma/informasi) dimodelkan melalui properti tingkat kesiapan teknologi (TRL) atau kompleksitas model data kecerdasan buatan yang disematkan ke dalam artefak.
- **Value Energy** (nilai ekonomis/sosial) dimodelkan melalui properti biaya per penumpang, harga tiket, atau penghematan siklus hidup.
- **Environmental Space Energy** (dampak lingkungan/ruang) dimodelkan melalui properti jejak karbon (emisi CO₂), efisiensi lahan, hingga tingkat polusi suara.
- **Relasi (Relationships) sebagai Konversi Transaksional:** Salah satu karakteristik utama Artefak PSKVE adalah kemampuannya melakukan “Konversi Transaksional”, yaitu pertukaran atau transformasi antardimensi PSKVE. Di dalam ontologi, proses ini diwujudkan melalui **Relasi** berwujud aturan logika (*rules*) yang saling mengaitkan satu properti dengan properti lain secara multi-domain. Contoh interaksi transaksional ini adalah bagaimana mesin menyimulasikan bahwa keberadaan *Knowledge Energy* (perangkat lunak cerdas untuk baterai EV) akan memberikan efek pada *Product Energy* (kinerja mesin yang lebih efisien), yang secara logis memiliki relasi langsung pada perbaikan *Environmental Space Energy* (emisi CO₂ yang lebih rendah) dan *Value Energy* (total biaya operasional yang lebih murah).

Secara keseluruhan, pemanfaatan ontologi berfungsi sebagai fondasi bahasa formal yang memastikan seluruh aspek multidimensional dan interdisipliner dari Artefak PSKVE tidak hanya menjadi konsep bisnis di atas kertas, tetapi memiliki struktur data riil yang bisa dinalar, dioptimalkan, dan dieksekusi secara terkomputasi oleh kecerdasan buatan.

2.1. Product (Produk)

Dalam paradigma *Smart Engineering*, luaran atau hasil akhir dari proses rekayasa disebut sebagai **Artefak PSKVE** (*Product, Service, Knowledge*,

Value, Environment), yaitu artefak yang cerdas atau memiliki komponen cerdas yang tersemat di dalamnya.

Di dalam kerangka energi multidimensional ini, **Product (Produk)** direpresentasikan secara lebih spesifik sebagai **Product Energy (Energi Produk)**. Sumber mendefinisikan *Product Energy* sebagai energi fisik atau alamiah, baik yang masih berbentuk bawaan maupun yang telah dikonversi, yang berada di dalam sebuah produk fisik. Contoh nyata dari wujud *Product Energy* ini adalah energi kimia yang tersimpan di dalam sebuah baterai atau energi kinetik penggerak yang dihasilkan oleh sebuah mesin.

Dalam konteks Artefak PSKVE yang lebih luas, “Produk” tidak berdiri sendiri, melainkan berinteraksi secara dinamis dengan empat dimensi energi lainnya melalui proses yang disebut sebagai **Konversi Transaksional (Transactional Conversion)**. Berikut adalah wujud interaksi energi produk dalam sistem rekayasa:

- **Interaksi dengan Dimensi Value (Nilai) dan Service (Layanan):** Dalam perekonomian *Smart Engineering*, interaksi pengguna selalu melibatkan pertukaran antardimensi energi ini. Sebagai contoh, seorang konsumen menukarkan energi finansial (*Value Energy*) untuk membeli sebuah perangkat pintar, yang memberikannya akses terhadap bentuk fisik alat tersebut (*Product Energy*) beserta teknologi cerdas di dalamnya (*Knowledge Energy*). Contoh lain pada transportasi, seorang penumpang menukarkan uang tiket (*Value Energy*) untuk menikmati perpindahan lokasi (*Service Energy*) melalui operasional sebuah kendaraan fisik (*Product Energy*).
- **Interaksi dengan Dimensi Knowledge (Pengetahuan) dan Environment (Lingkungan):** Dimensi pengetahuan sangat krusial dalam memaksimalkan potensi sebuah produk. Pengetahuan yang disematkan ke dalam produk—seperti algoritma pintar pada sistem baterai EV—secara langsung akan meningkatkan performa dan efisiensi dari *Product Energy* itu sendiri. Tingkat efisiensi energi produk fisik inilah yang pada akhirnya menentukan seberapa besar dampak artefak tersebut terhadap *Environmental Space Energy* (misalnya penekanan tingkat emisi gas buang CO₂).

Dari kacamata perancangan sistem rekayasa (Domain Sistem dan Teknologi), dimensi “Produk” menjadi fokus optimalisasi mesin cerdas. Dalam Domain Sistem, entitas kendaraan dipandang sebagai agen yang bertugas mengelola ketersediaan *Product Energy* (seperti bahan bakar atau listrik). Sementara pada Domain Teknologi, sebuah *smart engine* bertugas menjalankan siklus cerdasnya (PUDAL) untuk mencari cara paling efisien dalam mengonversi *Product Energy* tersebut menjadi energi kerja mekanis murni.

2. Artefak PSKVE

Singkatnya, dalam *Smart Engineering*, sebuah “Produk” tidak lagi dilihat sekadar sebagai cangkang material fisik, melainkan sebagai wadah pengonversi energi (*Product Energy*) yang secara terus-menerus bertransaksi dan saling memengaruhi dengan layanan yang diberikan, pengetahuan yang ditanamkan, nilai ekonomis yang dihasilkan, dan lingkungan di sekitarnya.

2.2. Service (Layanan)

Dalam kerangka rekayasa *Smart Engineering*, hasil akhir dari sebuah perancangan disebut sebagai **Artefak PSKVE** (*Product, Service, Knowledge, Value, Environment*), yaitu artefak yang terintegrasi dengan kecerdasan agar mampu berinteraksi secara cerdas dengan penggunanya maupun lingkungannya.

Di dalam model multi-dimensi energi ini, **Service (Layanan)** secara spesifik dikonseptualisasikan sebagai **Service Energy (Energi Layanan)**. Energi Layanan didefinisikan sebagai energi yang dikeluarkan atau ditangkap melalui proses penyediaan dan konsumsi sebuah layanan, yang secara kuat berkaitan dengan **metrik perhatian (attention), waktu, dan tingkat interaksi**. Contoh wujud nyata dari dimensi ini adalah seberapa lama waktu dan perhatian yang diinvestasikan pengguna saat berinteraksi dengan sebuah platform digital.

Dalam konteks Artefak PSKVE yang lebih luas, dimensi Layanan selalu terikat dalam proses dinamis yang disebut **Konversi Transaksional**, di mana ia saling bertukar wujud dengan dimensi energi lainnya:

- **Interaksi dengan Value (Nilai) dan Product (Produk):** Dalam simulasi seperti transportasi publik, seorang penumpang melakukan konversi transaksional dengan menukarkan uang tiket (*Value Energy*) untuk bisa mendapatkan efisiensi perpindahan lokasi (*Service Energy*), yang difasilitasi oleh operasional mesin kendaraan fisik (*Product Energy*).
- **Interaksi dengan Knowledge (Pengetahuan):** Pada jenis layanan digital, pengguna mungkin mengorbankan waktu dan perhatian interaktif mereka (*Service Energy*) untuk dapat mengekstraksi informasi yang dibutuhkan (*Knowledge Energy*) atau mendapatkan hiburan (*Value Energy*).

Agar sistem mesin atau model ontologi dapat mengevaluasi dan membandingkan kualitas “Layanan” dari suatu artefak cerdas, konsep layanan ini harus dikuantifikasi ke dalam properti yang terukur.

- **Efisiensi Waktu:** Dalam studi simulasi transportasi rute Bandung-Jakarta, Energi Layanan secara utama diukur menggunakan **estimasi waktu tempuh perjalanan**. Sebagai contoh, moda Kereta Cepat (*High-Speed Train*) tercatat menawarkan performa terbaik dari aspek Energi

Layanan karena mampu memberikan efisiensi waktu perjalanan tercepat bagi penumpangnya.

- **Kualitas Interaksi:** Untuk pengembangan model ontologi yang lebih mutakhir di masa depan, kuantifikasi dimensi *Service Energy* disarankan untuk diperluas ke berbagai metrik kualitas, seperti **tingkat keandalan sistem (MTBF - Mean Time Between Failures)**, **indeks kenyamanan pengguna**, **serta seberapa mudah layanan tersebut diakses (accessibility) oleh beragam kelompok demografi pengguna**.

Singkatnya, dimensi Layanan di dalam Artefak PSKVE menempatkan waktu, interaksi, dan kenyamanan manusia sebagai bentuk energi berharga. Sistem *Smart Engineering* bertujuan merancang mesin dan algoritma agar proses transformasi dari energi fisik produk menjadi layanan ini bisa berjalan seefisien dan sebaik mungkin.

2.3. Knowledge (Pengetahuan)

Dalam kerangka rekayasa *Smart Engineering*, luaran atau hasil dari sebuah perancangan disebut sebagai **Artefak PSKVE** (*Product, Service, Knowledge, Value, Environment*) yang dirancang untuk memiliki komponen cerdas di dalamnya. Di dalam model energi multidimensional ini, **Knowledge (Pengetahuan)** secara spesifik dikonseptualisasikan sebagai **Energi Pengetahuan (Knowledge Energy)**.

Definisi dan Wujud Energi Pengetahuan Energi Pengetahuan didefinisikan sebagai energi intelektual atau informasional yang terwujud dalam bentuk skema, algoritma, kepakaran/keahlian, serta pengetahuan operasional (“*how-to*”). Contoh nyata dari wujud energi pengetahuan ini adalah logika atau struktur instruksi di dalam sebuah model Kecerdasan Buatan (AI). Pada tingkat Teknologi (Domain Teknologi), *Knowledge Energy* inilah yang diintegrasikan ke dalam sebuah *smart engine* (mesin cerdas) agar mesin tersebut mampu menjalankan siklus PUDAL dan mengoptimalkan operasinya.

Konversi Transaksional dengan Dimensi PSKVE Lainnya Sama seperti dimensi produk atau layanan, *Knowledge Energy* di dalam artefak dirancang untuk secara konstan berinteraksi dan bertransformasi dengan dimensi energi lainnya melalui proses “Konversi Transaksional”:

- **Meningkatkan Kualitas Produk dan Lingkungan:** Pengetahuan cerdas yang disematkan ke dalam sistem secara langsung mengendalikan performa mesin. Sebagai contoh simulasi transportasi, *Knowledge Energy* yang tertanam dalam algoritma teknologi baterai kendaraan listrik (EV) atau pada sistem manajemen *smart grid* (jaringan cerdas) akan secara

2. Artefak PSKVE

langsung mengefisienkan *Product Energy* (konsumsi daya fisik). Efisiensi tersebut secara logis akan menekan dampak emisi gas buang, yang berarti memperbaiki kualitas *Environmental Space Energy*.

- **Bertransaksi dengan Layanan dan Nilai:** Dari kacamata ekonomi dan interaksi pengguna, konsumen menukarkan biaya finansial (*Value Energy*) untuk mendapatkan perangkat fisik (*Product Energy*) beserta Energi Pengetahuan yang tertanam di dalamnya. Selain itu, pengguna kerap menginvestasikan waktu serta atensi mereka (*Service Energy*) untuk menggali informasi dari sistem cerdas tersebut (*Knowledge Energy*). Tantangan riset ke depan adalah mencari model matematis konversi dari seberapa besar “energi pengetahuan” pada sebuah perangkat lunak bisa secara efektif dikonversi menjadi “energi nilai” komersial (*Value Energy*).

Kuantifikasi dalam Pemodelan (Ontologi) Agar *Knowledge Energy* ini dapat dimasukkan, diukur, dan disimulasikan oleh mesin penalaran (seperti pada model ontologi Prolog), dimensi abstrak ini harus diurai menjadi kumpulan metrik atau properti yang dapat dikuantifikasi. Beberapa properti pengukur tingkat *Knowledge Energy* pada suatu artefak meliputi .:

- **TRL (Technology Readiness Level / Tingkat Kesiapan Teknologi):** Seberapa matang pengetahuan teknis tersebut bisa diaplikasikan.
- **Kompleksitas Algoritma:** Tingkat kerumitan dan kapabilitas komputasi dari perangkat lunak yang mendasarinya.
- **Kebutuhan Data (Data Requirements):** Kuantitas dan kualitas data yang diwajibkan agar komponen sistem AI bisa berfungsi optimal.

Singkatnya, di dalam konsep Artefak PSKVE, “Pengetahuan” bukan sekadar buku panduan manual, melainkan diperlakukan sebagai **energi intelektual (seperti algoritma dan AI) penggerak utama mesin** yang menjembatani efisiensi performa perangkat fisik, nilai ekonomi pengguna, hingga keamanan bagi lingkungan sekitarnya.

2.4. Value (Nilai)

Dalam kerangka desain *Smart Engineering*, luaran atau hasil akhir dari proses rekayasa disebut sebagai **Artefak PSKVE** (*Product, Service, Knowledge, Value, Environment*). Di dalam model energi multidimensional ini, dimensi **Value (Nilai)** dikonseptualisasikan secara khusus sebagai **Energi Nilai (Value Energy)**.

Definisi Energi Nilai *Value Energy* merepresentasikan **potensi atau kelayakan nyata (realized worth)** dari sebuah artefak, yang wujudnya

tidak terbatas pada metrik uang atau finansial, tetapi juga meluas pada nilai sosial, budaya, hingga reputasi. Contoh nyata dari perwujudan dimensi nilai ini adalah tingkat kapitalisasi pasar dari suatu produk atau tingkat pengaruh sosial yang dihasilkannya di masyarakat.

Konversi Transaksional dengan Dimensi PSKVE Lainnya Di dalam sebuah Artefak PSKVE, Energi Nilai bersifat dinamis karena terus-menerus berinteraksi dan bertukar wujud dengan dimensi-dimensi lainnya melalui mekanisme “Konversi Transaksional”.

- **Interaksi Produk, Pengetahuan, dan Layanan:** Konsumen melakukan konversi ini dalam kehidupan sehari-hari, contohnya dengan menukarkan uang (*Value Energy*) untuk membeli sebuah perangkat keras pintar (*Product Energy*) yang digerakkan oleh algoritma cerdas (*Knowledge Energy*).- Pada contoh layanan transportasi, penumpang secara harfiah menukarkan uang mereka (*Value Energy*) demi mendapatkan jasa efisiensi waktu perjalanan (*Service Energy*) melalui pemanfaatan mesin kendaraan tersebut (*Product Energy*).- Selain itu, saat pengguna menggunakan sebuah platform digital, mereka menginvestasikan waktu dan perhatian (*Service Energy*) untuk mengekstraksi informasi (*Knowledge Energy*) atau sekadar mendapatkan hiburan dan manfaat lainnya (*Value Energy*).

Kuantifikasi dan Metrik Evaluasi Agar *Value Energy* ini dapat dimasukkan ke dalam model ontologi (seperti Prolog) dan dioptimalkan oleh kecerdasan buatan, nilai abstrak ini harus diterjemahkan menjadi metrik yang terukur secara komputasional:

- **Total Biaya:** Pada studi simulasi perbandingan transportasi rute Bandung-Jakarta tahun 2030, *Value Energy* dievaluasi dengan menggunakan metrik **total biaya per penumpang**. Ini dikalkulasi dengan mengakumulasi seluruh biaya energi/bahan bakar, tarif jalan tol, alokasi biaya perawatan tahunan (untuk moda seperti bus atau mobil listrik pribadi), atau menggunakan harga tiket langsung (untuk kereta cepat).
- **Perluasan Metrik Ekonomi dan Sosial:** Untuk perancangan artefak cerdas yang lebih komprehensif di masa depan, sumber-sumber menyarankan agar pengukuran dimensi Nilai ini diperluas. Metrik yang diusulkan mencakup **perhitungan Biaya Siklus Hidup yang utuh (Full Lifecycle Costing / LCC)**, analisis biaya-manfaat bagi masyarakat, **probabilitas penciptaan lapangan kerja**, serta seberapa besar dampak artefak tersebut terhadap perkembangan ekonomi lokal.

Salah satu fokus riset fundamental di masa depan dalam kerangka *Smart Engineering* adalah mengembangkan pemodelan formal untuk mengukur tingkat konversi antar-dimensi ini. Sebagai contoh, para peneliti harus merumuskan

2. Artefak PSKVE

bagaimana metrik abstrak dari *Knowledge Energy* (seperti kebaruan perangkat lunak buatan insinyur) dapat dikonversi secara efektif untuk menjadi *Value Energy* (seperti keuntungan finansial) yang dirasakan langsung oleh penggunanya.

2.5. Environment (Lingkungan)

Dalam paradigma *Smart Engineering*, luaran akhir dari proses rekayasa adalah **Artefak PSKVE** (*Product, Service, Knowledge, Value, Environment*), di mana dimensi **Environment (Lingkungan)** dikonseptualisasikan secara khusus sebagai **Environmental Space Energy (Energi Ruang Lingkungan)**.

Definisi dan Wujud Energi Ruang Lingkungan Berbeda dengan sekadar lingkungan fisik tempat sebuah benda berada, Energi Ruang Lingkungan merujuk pada **daya pengaruh dari sebuah ruang atau lingkungan, yang mencakup kualitas visioner, imersif, dan inspirasionalnya, serta jejak ekologis yang ditimbulkannya**. Wujud dari dimensi energi ini bisa berupa hal yang bersifat membangun—seperti desain ruang kerja kolaboratif yang mampu menumbuhkan kreativitas penggunanya—maupun hal yang berupa dampak (*environmental footprint*) seperti jejak emisi karbon yang ditinggalkan oleh sistem operasional.

Konversi Transaksional dengan Dimensi PSKVE Lainnya Di dalam sebuah sistem cerdas multi-domain, Energi Lingkungan tidak berdiri sendiri; ia saling memengaruhi dan ditransformasikan bersama dimensi energi lainnya melalui proses “Konversi Transaksional”.

- **Interaksi dengan Pengetahuan (Knowledge) dan Produk (Product):** *Environmental Space Energy* sangat erat kaitannya dengan inovasi teknologi yang tertanam pada artefak. Sebagai contoh nyata, Energi Pengetahuan berupa algoritma kecerdasan buatan pada teknologi baterai kendaraan listrik (EV) atau manajemen jaringan cerdas (*smart grid*) akan secara langsung mengendalikan efisiensi konsumsi Energi Produk (kinerja perangkat keras). Peningkatan efisiensi mesin fisik tersebut secara logis akan menekan dampak gas buang CO₂ secara drastis, yang berarti kualitas dari Energi Ruang Lingkungan menjadi jauh lebih baik.

Kuantifikasi dan Metrik Evaluasi dalam Ontologi Agar keberadaan Lingkungan dapat dihitung dan dioptimalkan oleh mesin penalar cerdas (seperti menggunakan ontologi basis pengetahuan Prolog digabungkan dengan Python), dimensi abstrak ini harus diukur ke dalam properti yang konkret:

- **Emisi Karbon (CO₂):** Pada contoh studi kasus simulasi pemilihan transportasi Bandung-Jakarta 2030, kualitas Lingkungan dikuantifikasi

secara utama melalui metrik **emisi CO2 ekuivalen per penumpang**. Hasil pemodelan mesin mampu menyajikan perbandingan bahwa opsi moda seperti Kereta Cepat Listrik memberikan performa perlindungan Lingkungan yang lebih superior dengan menghasilkan tingkat emisi per penumpang terendah, dibandingkan pemakaian armada Bus Diesel yang jauh lebih berpolusi.

- **Perluasan Metrik Holistik di Masa Depan:** Agar perancangan artefak PSKVE menjadi lebih presisi di masa mendatang, pengembangan model evaluasi untuk dimensi *Environmental Space Energy* disarankan tidak hanya berhenti di emisi karbon. Kerangka kerja ini mengusulkan metrik yang lebih luas, seperti **penilaian polusi suara (noise pollution)**, **efisiensi tata guna lahan (land use efficiency)**, **dampak visual terhadap ruang**, **hingga kalkulasi jejak emisi siklus hidup secara utuh—dari tahap awal manufaktur pembuatan artefak hingga fase pembuangannya**.

Secara keseluruhan, pemosisian Lingkungan sebagai suatu “Energi” dalam Artefak PSKVE merevolusi pandangan desain rekayasa. Sebuah sistem dikatakan benar-benar cerdas jika mesinnya tidak sekadar mengejar performa teknis (Produk) dan efisiensi waktu (Layanan), tetapi secara sadar mengoptimalkan Pengetahuan komputasionalnya untuk menjaga serta menciptakan kualitas Ruang Lingkungan hidup yang keberlanjutan secara holistik.

3

Energi Multi-Dimensi

Dalam paradigma *Smart Engineering*, konsep **Energi Multi-Dimensi** memperluas pandangan tradisional tentang energi—yang biasanya murni bersifat fisik (seperti energi kimia atau kinetik)—menjadi lima dimensi holistik yang disebut **PSKVE (Product, Service, Knowledge, Value, Environment)**.

Agar mesin komputasi (kecerdasan buatan) dapat mengevaluasi, memproses, dan menyimulasikan kelima bentuk energi ini, abstraksi energi tersebut harus dipetakan ke dalam struktur data formal menggunakan **Konsep Inti Ontologi**. Integrasi ini memungkinkan teori energi multidimensi direpresentasikan melalui empat bangunan dasar ontologi, yaitu:

- **Properti (Properties) sebagai Pengukur Dimensi Energi:** Di dalam basis pengetahuan ontologis (seperti pada bahasa Prolog), abstraksi dari setiap dimensi energi PSKVE diterjemahkan secara langsung menjadi atribut atau *Properti* matematis yang terukur. Sebagai contoh pemodelan pada transportasi:
- **Product Energy** dikuantifikasi sebagai properti teknis seperti konsumsi daya listrik atau bahan bakar.
- **Service Energy** (yang terkait dengan waktu dan interaksi) diukur melalui properti estimasi waktu tempuh perjalanan.
- **Value Energy** (nilai ekonomis) direpresentasikan sebagai properti total biaya operasional per penumpang atau harga tiket.

3. Energi Multi-Dimensi

- **Environmental Space Energy** dimodelkan melalui properti jejak emisi CO2 ekuivalen.
- **Kelas (Classes) dan Individu (Individuals) sebagai Wadah Energi:** Entitas fisik maupun sistem yang membawa dan mengonversi energi multi-dimensi ini didefinisikan sebagai **Individu** yang bernaung di bawah suatu **Kelas**. Misalnya, di dalam sistem ontologi, “EV_Sedan” atau “High-Speed Electric Train” dideklarasikan secara eksplisit sebagai individu spesifik dari kelas “Kendaraan”. Mesin menggunakan deklarasi ini untuk mengenali spesifikasi kapasitas energi dari masing-masing kendaraan tersebut.
- **Relasi (Relationships) sebagai Model Konversi Transaksional:** Salah satu karakteristik sentral dari Energi Multi-Dimensi adalah kemampuannya untuk saling bertukar dan mentransformasikan wujud antardimensi, sebuah proses yang disebut **Konversi Transaksional**. Di dalam ontologi, pertukaran energi lintas dimensi ini dimodelkan menggunakan **Relasi** dan aturan logika (*rules*). Ontologi memfasilitasi mesin untuk menalar logika interaksi kompleks ini. Sebagai contoh, sistem dapat menyimulasikan relasi logika di mana peningkatan **Knowledge Energy** (seperti perbaikan algoritma pintar pada manajemen baterai) akan memengaruhi efisiensi **Product Energy** (perangkat keras mesin), yang mana relasi tersebut secara otomatis akan menghasilkan dampak turunan pada penurunan emisi gas buang (**Environmental Energy**).

Pemanfaatan konsep inti ontologi di sini bertindak sebagai fondasi operasional yang vital. Tanpa ontologi, Energi Multi-Dimensi hanya akan menjadi konsep teoretis. Namun dengan ontologi, kelima dimensi energi ini berhasil diubah menjadi parameter komputasional terstruktur yang memungkinkan perangkat lunak untuk menalar efisiensi, mengevaluasi *trade-off* (tarik-ulur) antar-energi, dan merancang solusi teknik interdisipliner secara otomatis.

3.1. Energi Produk (Fisik)

Dalam kerangka *Smart Engineering*, konsep Energi Multi-Dimensi memperluas pandangan tradisional—yang sebelumnya hanya melihat energi murni dari kacamata fisik—menjadi lima dimensi holistik yang disebut PSKVE (*Product, Service, Knowledge, Value, Environment*). Di dalam model ini, **Energi Produk (Product Energy)** didefinisikan sebagai wujud energi alamiah atau fisik, baik yang masih bawaan maupun yang telah dikonversi, yang melekat di dalam sebuah produk. Contoh nyata dari keberadaan Energi Produk adalah energi kimia yang tersimpan di dalam sel baterai atau energi kinetik yang menggerakkan suatu mesin.

Sebagai bagian dari ekosistem Energi Multi-Dimensi, **Energi Produk** tidak beroperasi secara terisolasi, melainkan secara konstan saling bertukar dan bertransformasi dengan dimensi energi lainnya melalui mekanisme yang disebut “Konversi Transaksional”. Keterkaitan ini terlihat jelas melalui interaksi berikut:

- **Integrasi dengan Pengetahuan (Knowledge) dan Lingkungan (Environment):** Energi Pengetahuan (*Knowledge Energy*), seperti algoritma cerdas pada manajemen baterai kendaraan listrik (EV) atau jaringan cerdas (*smart grid*), secara langsung mengendalikan dan mengoptimalkan efisiensi konsumsi Energi Produk pada perangkat keras. Seberapa efisien Energi Produk ini dikelola oleh kecerdasan mesin akan berimbas langsung pada kualitas Energi Ruang Lingkungan, seperti seberapa besar jejak emisi karbon yang dihasilkan.
- **Pertukaran dengan Nilai (Value) dan Layanan (Service):** Dari kacamata ekonomi dan pengguna, sebuah konversi transaksional terjadi ketika konsumen menukarkan uang finansial mereka (Energi Nilai) untuk memperoleh perangkat pintar yang di dalamnya memuat wujud fisik alat tersebut (Energi Produk) beserta algoritma cerdasnya (Energi Pengetahuan). Pada contoh transportasi, penumpang membayar tiket (Energi Nilai) demi mendapatkan akses terhadap operasional kendaraan fisik tersebut (Energi Produk) untuk menghasilkan efisiensi waktu perjalanan (Energi Layanan).

Dalam pemecahan masalah rekayasa (seperti metafora sistem transportasi lintas domain), pengelolaan Energi Produk menjadi fokus operasional utama pada tingkatan teknis:

- **Pada Domain Sistem:** Entitas utama seperti kendaraan dipandang sebagai sistem cerdas yang bertugas menavigasi lingkungan sekaligus secara aktif mengelola ketersediaan Energi Produk (seperti konsumsi bahan bakar atau daya listrik) miliknya.
- **Pada Domain Teknologi:** Komponen inti seperti mesin cerdas (*Smart Engine*) memanfaatkan siklus PUDAL untuk mengoptimalkan konversi teknologi, yakni berfokus pada cara mengubah Energi Produk menjadi energi kerja (mekanis) yang efisien bagi kendaraan tersebut.

Singkatnya, meskipun Energi Produk merupakan representasi daya fisik yang menjadi motor penggerak material dari suatu artefak, kerangka energi multidimensional memosisikannya sebagai komponen yang harus dikendalikan oleh “pengetahuan” algoritmik demi mencapai efisiensi tertinggi, sekaligus menghasilkan “layanan” dan “nilai” yang optimal bagi pengguna serta “lingkungan”.

3.2. Energi Layanan (Atensi)

Dalam kerangka *Smart Engineering*, konsep **Energi Multi-Dimensi** memperluas pandangan tradisional tentang energi (yang biasanya hanya merujuk pada energi fisik) menjadi lima dimensi holistik yang disebut **PSKVE** (*Product, Service, Knowledge, Value, Environment*). Di dalam model ini, **Energi Layanan (Service Energy)** dikonseptualisasikan secara khusus sebagai bentuk energi yang berkaitan erat dengan **waktu, interaksi, dan atensi (perhatian) manusia**.

Definisi dan Wujud Energi Layanan Energi Layanan didefinisikan sebagai energi yang dikeluarkan atau ditangkap melalui proses penyediaan maupun konsumsi sebuah layanan. Wujud nyata dari dimensi energi ini tidak berupa daya mekanis, melainkan keterlibatan dari sisi manusia. Sebagai contoh, durasi waktu dan tingkat perhatian (*user engagement*) yang diinvestasikan oleh seseorang saat berinteraksi dengan sebuah platform digital merupakan wujud langsung dari Energi Layanan.

Konversi Transaksional dengan Dimensi Energi Lainnya Sebagai bagian dari Energi Multi-Dimensi, Energi Layanan bersifat dinamis dan terus-menerus bertukar wujud dengan dimensi energi lainnya melalui proses yang disebut “Konversi Transaksional”. Interaksi ini terlihat jelas dalam skenario berikut:

- **Pertukaran dengan Nilai (Value) dan Produk (Product):** Dalam simulasi sistem transportasi publik, sebuah konversi transaksional terjadi ketika penumpang menukarkan uang finansial mereka (Energi Nilai) untuk memperoleh jasa perpindahan lokasi (Energi Layanan), yang mana layanan tersebut difasilitasi oleh operasional mesin kendaraan fisik (Energi Produk).
- **Pertukaran dengan Pengetahuan (Knowledge) dan Nilai (Value):** Pada sistem interaktif, seorang pengguna mengorbankan waktu dan perhatian interaktif mereka (Energi Layanan) untuk dapat mengekstraksi informasi dari suatu sistem (Energi Pengetahuan) atau sekadar untuk mendapatkan hiburan dan manfaat lainnya (Energi Nilai).

Kuantifikasi dan Evaluasi dalam Ontologi Sistem Cerdas Agar sistem penalaran mesin (seperti platform ontologi gabungan Prolog dan Python) dapat membandingkan dan mengoptimalkan kualitas dari sebuah layanan, abstraksi Energi Layanan ini harus diterjemahkan ke dalam bentuk properti atau metrik yang terukur:

- **Efisiensi Waktu (Estimasi Waktu Tempuh):** Pada studi kasus simulasi pemilihan alternatif transportasi rute Bandung-Jakarta 2030, kualitas Energi Layanan dievaluasi secara utama melalui **estimasi waktu tempuh**

perjalanan. Hasil simulasi menunjukkan bahwa moda Kereta Cepat Listrik (*High-Speed Train*) memberikan performa Energi Layanan yang paling superior karena menawarkan waktu perjalanan yang paling singkat bagi penumpangnya.

- **Perluasan Metrik Pengukuran di Masa Depan:** Untuk perancangan artefak cerdas yang lebih komprehensif ke depannya, kerangka kerja ini menyarankan agar pengukuran Energi Layanan diperluas. Metrik yang diusulkan untuk mengkuantifikasi dimensi ini meliputi **pengukuran tingkat keandalan sistem (MTBF - Mean Time Between Failures), indeks kenyamanan pengguna, hingga seberapa mudah layanan tersebut dapat diakses (accessibility) oleh beragam kelompok demografi.**

Singkatnya, penyertaan Energi Layanan di dalam konsep Energi Multi-Dimensi menegaskan bahwa dalam paradigma rekayasa modern, waktu dan perhatian manusia diperlakukan sebagai wujud “energi” yang berharga. Sistem mesin yang cerdas tidak lagi hanya dirancang untuk menghemat bahan bakar fisik (Produk), tetapi juga untuk memastikan bahwa interaksi layanannya terhadap pengguna dapat berjalan seefisien dan menyenangkan mungkin.

3.3. Energi Pengetahuan (Algoritma)

Dalam kerangka *Smart Engineering*, konsep **Energi Multi-Dimensi** memperluas definisi energi dari yang sebelumnya sekadar entitas fisik mekanis menjadi lima dimensi holistik yang disebut **PSKVE** (*Product, Service, Knowledge, Value, Environment*). Di dalam model komprehensif ini, **Energi Pengetahuan (Knowledge Energy)** secara spesifik didefinisikan sebagai energi intelektual atau informasional yang terwujud dalam bentuk skema, keahlian kepakaran, hingga algoritma dan logika di dalam sebuah model Kecerdasan Buatan (AI).

Sebagai bagian integral dari ekosistem Energi Multi-Dimensi, Energi Pengetahuan tidak beroperasi secara terisolasi. Alih-alih, ia berfungsi sebagai motor penggerak “kecerdasan” di dalam sistem yang terus-menerus bertukar wujud dan memengaruhi dimensi energi lainnya melalui mekanisme **Konversi Transaksional**. Interaksi multidimensional algoritma ini terlihat pada proses berikut:

- **Mengendalikan Energi Produk dan Memperbaiki Energi Lingkungan:** Pada tingkat Teknologi, sebuah mesin cerdas (*Smart Engine*) memanfaatkan siklus PUDAL dengan mengintegrasikan Energi Pengetahuan (algoritma) untuk mengoptimalkan konversi Energi Produk (sumber daya

3. Energi Multi-Dimensi

fisik seperti bahan bakar) menjadi energi kerja yang efisien. Sebagai contoh, keberadaan *Knowledge Energy* yang tertanam dalam algoritma teknologi baterai kendaraan listrik (EV) atau pada sistem manajemen *smart grid* (jaringan cerdas) akan secara langsung memengaruhi efisiensi *Product Energy*. Peningkatan efisiensi operasional sistem keras fisik ini secara otomatis akan memberikan dampak turunan yang positif terhadap *Environmental Space Energy*, seperti penurunan drastis pada tingkat emisi karbon (CO₂).

- **Bertransaksi dengan Energi Nilai dan Layanan:** Dari sudut pandang pengguna dan ekonomi, sebuah konversi transaksional terjadi ketika konsumen menukarkan biaya finansial atau uang (*Value Energy*) demi mendapatkan sebuah perangkat cerdas yang di dalamnya memuat wujud keras fisik (*Product Energy*) beserta Energi Pengetahuan (algoritma cerdas). Selain itu, pengguna sering kali harus mengorbankan waktu dan tingkat perhatian mereka (*Service Energy*) untuk dapat berinteraksi dan mengekstraksi informasi berharga dari sistem cerdas tersebut (*Knowledge Energy* atau *Value Energy*).

Kuantifikasi dalam Pemodelan (Ontologi Sistem Cerdas) Agar keberadaan Energi Pengetahuan ini dapat dievaluasi, disimulasikan, dan dioptimalkan oleh mesin penalaran komputasi (seperti pada implementasi platform hibrida Prolog-Python), dimensi abstrak dari sebuah algoritma harus diterjemahkan menjadi metrik properti yang dapat dikuantifikasi. Pengembangan model *Smart Engineering* mengusulkan agar Energi Pengetahuan diukur menggunakan parameter seperti:

- **Tingkat Kesiapan Teknologi (TRL - Technology Readiness Level).**
- **Tingkat kompleksitas dari algoritma yang mendasari sistem tersebut.**
- **Kebutuhan data (data requirements) yang diwajibkan agar komponen kecerdasan buatan (AI) dapat berfungsi secara maksimal.**

Singkatnya, di dalam konsep Energi Multi-Dimensi, algoritma diposisikan bukan sekadar sebagai barisan kode pelengkap, melainkan diperlakukan sebagai **energi intelektual utama** yang mampu mengorkestrasi interaksi fungsional ke seluruh domain. *Knowledge Energy* inilah yang memastikan bahwa energi fisik beroperasi pada tingkat efisiensi maksimal untuk memberikan layanan terbaik, nilai ekonomis yang tinggi, sembari menjaga keseimbangan dan keberlanjutan energi lingkungan sekitarnya.

3.4. Energi Nilai (Finansial/Sosial)

Dalam paradigma *Smart Engineering*, energi tidak lagi hanya dipandang sebagai entitas fisik, melainkan diperluas menjadi **Energi Multi-Dimensi** yang mencakup lima elemen PSKVE (*Product, Service, Knowledge, Value, Environment*). Di dalam model holistik ini, **Energi Nilai (Value Energy)** secara khusus didefinisikan sebagai **potensi atau kelayakan nyata (realized worth)** dari sebuah artefak atau sistem.

Definisi dan Wujud Energi Nilai Karakteristik utama dari Energi Nilai adalah wujudnya yang sangat luas. Dimensi ini **tidak hanya sebatas pada ukuran finansial atau metrik uang, tetapi juga mencakup nilai sosial, budaya, hingga reputasi**. Contoh nyata dari perwujudan Energi Nilai ini adalah tingkat kapitalisasi pasar yang dimiliki oleh suatu perusahaan atau besarnya pengaruh sosial (*social influence*) yang dihasilkan oleh sebuah inovasi.

Konversi Transaksional dengan Dimensi Energi Lainnya Sebagai bagian dari ekosistem Energi Multi-Dimensi, Energi Nilai bersifat sangat dinamis dan berinteraksi secara terus-menerus dengan dimensi lainnya melalui proses **Konversi Transaksional**. Interaksi ini dapat diamati dalam berbagai aktivitas kehidupan:

- **Pertukaran Finansial dan Teknologi:** Seorang konsumen melakukan konversi energi ketika ia menukarkan uang finansial yang dimilikinya (Energi Nilai) untuk membeli sebuah perangkat pintar (Energi Produk) yang di dalamnya telah tertanam algoritma kecerdasan (Energi Pengetahuan).
- **Pertukaran Waktu dan Manfaat:** Seorang pengguna mungkin menginvestasikan waktu dan perhatiannya (Energi Layanan) saat berinteraksi dengan sebuah sistem digital untuk mendapatkan hiburan atau manfaat lain yang bernilai baginya (Energi Nilai/Pengetahuan).

Dalam konteks fundamental riset ke depan, tantangan utamanya adalah bagaimana mengoptimalkan konversi transaksional ini, misalnya meneliti seberapa efektif inovasi perangkat lunak (Energi Pengetahuan) dapat dikonversi menjadi keuntungan yang nyata (Energi Nilai).

Kuantifikasi Energi Nilai dalam Simulasi Sistem Cerdas Agar Energi Nilai ini dapat diproses, dievaluasi, dan disimulasikan oleh mesin (seperti implementasi ontologi berbasis bahasa Prolog dan Python), nilai abstrak ini harus diterjemahkan menjadi metrik properti yang konkret.

Dalam studi kasus simulasi pemilihan alternatif transportasi rute Bandung-Jakarta 2030, kualitas **Energi Nilai** diukur secara langsung melalui **metrik Total Biaya per Penumpang**.

3. Energi Multi-Dimensi

- Untuk moda seperti armada bus diesel atau mobil listrik (EV) pribadi, perhitungan Energi Nilai ini mencakup akumulasi biaya bahan bakar/listrik, tarif jalan tol, hingga perhitungan pembagian pro-rata dari biaya perawatan tahunan kendaraan.- Sementara itu, untuk moda transportasi seperti Kereta Cepat, harga tiket penumpang digunakan sebagai parameter pengukur Energi Nilai.

Perluasan Metrik Evaluasi Ekonomi dan Sosial di Masa Depan Meski simulasi tahap awal menggunakan metrik biaya langsung, sumber-sumber tersebut menekankan bahwa pengukuran biaya operasional semata tidak cukup untuk merepresentasikan keseluruhan “Nilai” dari suatu artefak. Agar analisis menjadi lebih komprehensif, pengembangan model pengukuran Energi Nilai di masa depan disarankan untuk mencakup metrik yang lebih holistik, seperti:

- **Biaya Siklus Hidup yang utuh (Full Lifecycle Costing / LCC)**, yang turut memperhitungkan biaya modal awal untuk pembuatan kendaraan dan infrastrukturnya.
- **Analisis biaya-manfaat kemasyarakatan (societal cost-benefit analysis).**
- **Penilaian potensi penciptaan lapangan kerja dan sejauh mana artefak tersebut memberikan dampak bagi perkembangan ekonomi lokal.**

Singkatnya, pemosisian Nilai sebagai sebuah dimensi energi dalam *Smart Engineering* menuntut para insinyur untuk tidak sekadar menciptakan produk yang berfungsi secara fisik, melainkan juga harus memastikan bahwa produk tersebut dirancang agar mampu menghasilkan kelayakan ekonomi, sosial, dan budaya tertinggi bagi masyarakat.

3.5. Energi Ruang Lingkungan (Inspirasi)

Dalam paradigma *Smart Engineering*, konsep **Energi Multi-Dimensi** memperluas definisi energi dari sekadar wujud fisik mekanis menjadi lima dimensi holistik yang dikenal sebagai **PSKVE** (*Product, Service, Knowledge, Value, Environment*). Di dalam kerangka komprehensif ini, **Energi Ruang Lingkungan (Environmental Space Energy)** secara spesifik didefinisikan sebagai daya pengaruh dari sebuah ruang atau lingkungan, yang mencakup kualitas visioner, imersif, dan inspirasionalnya.

Wujud Energi Ruang Lingkungan (Inspirasi dan Dampak) Berbeda dengan pandangan tradisional yang sekadar melihat lingkungan sebagai lokasi

fisik statis, dimensi ini memandang bahwa ruang memiliki “energi” yang secara aktif memengaruhi entitas di dalamnya. Wujud nyata dari dimensi ini bisa bersifat sangat membangun (inspirasional), seperti **desain dari sebuah ruang kerja kolaboratif yang terbukti mampu menumbuhkan kreativitas para penggunanya**. Di sisi lain, energi ini juga merepresentasikan dampak ekologis (*environmental footprint*) yang ditinggalkan oleh operasional suatu sistem, seperti jejak emisi karbon.

Konversi Transaksional dengan Dimensi Energi Lainnya Sebagai komponen integral dari ekosistem Energi Multi-Dimensi, Energi Ruang Lingkungan bersifat dinamis dan terus-menerus bertukar wujud dengan dimensi energi lainnya melalui proses yang disebut **Konversi Transaksional**.

- **Interaksi Erat dengan Pengetahuan (Knowledge) dan Produk (Product):** Kualitas dari Energi Ruang Lingkungan sangat dipengaruhi oleh tingkat kecerdasan buatan yang tertanam dalam suatu artefak. Sebagai contoh, Energi Pengetahuan yang terwujud dalam algoritma teknologi baterai kendaraan listrik (EV) atau sistem manajemen *smart grid* akan secara langsung mengendalikan dan mengoptimalkan Energi Produk (kinerja perangkat keras fisik). Optimalisasi efisiensi pada mesin fisik ini secara logis akan memberikan hasil turunan berupa perbaikan kualitas Energi Ruang Lingkungan, yakni dengan menekan dampak emisi gas buang secara signifikan.

Kuantifikasi dan Metrik Evaluasi dalam Ontologi Agar “energi ruang” yang abstrak ini dapat dievaluasi, disimulasikan, dan dioptimalkan oleh sistem penalaran mesin (seperti pada platform hibrida Prolog dan Python), ia harus diurai menjadi properti atau metrik yang dapat dihitung:

- **Pengukuran Jejak Emisi (CO₂):** Dalam studi simulasi pemilihan alternatif transportasi rute Bandung-Jakarta 2030, kualitas Energi Ruang Lingkungan secara praktis dikuantifikasi melalui metrik **emisi CO₂ ekuivalen per penumpang**, yang menarik langsung rekam jejak lingkungan dari operasional kendaraan.
- **Perluasan Metrik Holistik di Masa Depan:** Agar perancangan sistem rekayasa menjadi lebih presisi dalam menilai kualitas lingkungan dan ruang inspirasionalnya, pengembangan kerangka kerja di masa depan mengusulkan metrik pengukuran yang jauh lebih luas. Metrik tersebut mencakup **penilaian polusi suara (noise pollution)**, **efisiensi tata guna lahan**, **dampak visual dari suatu ruang**, **hingga kalkulasi jejak emisi siklus hidup secara utuh**—sejak tahap manufaktur artefak hingga fase pembuangannya.

3. Energi Multi-Dimensi

Singkatnya, pemosisian Ruang Lingkungan sebagai salah satu pilar Energi dalam model PSKVE menegaskan bahwa perancangan sebuah sistem rekayasa tidak boleh hanya berfokus pada kekuatan mekanis murni (Produk). Sebaliknya, sistem harus dirancang secara cerdas agar mampu melestarikan ekologi sekaligus menciptakan lingkungan yang kaya akan nilai inspirasional dan visioner bagi manusia di dalamnya.

3.6. Konversi Transaksional

Dalam kerangka *Smart Engineering*, konsep Energi Multi-Dimensi memperluas definisi energi dari sekadar bentuk fisik menjadi lima dimensi holistik yang disebut PSKVE (*Product, Service, Knowledge, Value, Environment*). Di dalam ekosistem energi yang komprehensif ini, **Konversi Transaksional (Transactional Conversion)** didefinisikan sebagai proses di mana satu bentuk energi PSKVE dipertukarkan atau ditransformasikan menjadi bentuk energi lain lintas dimensi.

Konsep konversi transaksional ini muncul sebagai konsekuensi dari perluasan paradigma rekayasa, dari yang sebelumnya hanya berfokus pada mesin (*machine-only paradigm*) menjadi paradigma gabungan antara manusia dan mesin (*human-machine paradigm*). Berbeda dengan konversi energi tradisional yang biasanya berfokus pada perubahan wujud di dalam satu dimensi fisik yang sama, **konversi transaksional secara khusus mengkaji pertukaran (transactions) energi antar-dimensi yang berbeda.**

Beberapa wujud nyata dari mekanisme **Konversi Transaksional** ini meliputi:

- **Pertukaran Nilai dengan Produk dan Pengetahuan:** Seorang konsumen melakukan konversi transaksional saat menukarkan energi finansial (*Value Energy*) untuk membeli sebuah perangkat pintar yang memuat komponen keras fisik (*Product Energy*) beserta teknologi atau algoritma yang tertanam di dalamnya (*Knowledge Energy*).
- **Pertukaran Layanan dengan Pengetahuan dan Nilai:** Seorang pengguna menginvestasikan waktu dan perhatiannya (*Service Energy*) saat berinteraksi dengan sebuah platform digital guna memperoleh informasi (*Knowledge Energy*) atau hiburan (*Value Energy*).
- **Pertukaran Transportasi:** Pada simulasi sistem transportasi, penumpang secara aktif melakukan konversi dengan membayarkan uang (*Value Energy*) demi mendapatkan layanan perpindahan lokasi yang efisien secara waktu (*Service Energy*) serta akses terhadap operasional kendaraan fisik tersebut (*Product Energy*).

Dalam perancangan sistem rekayasa modern, pemahaman dan optimalisasi konversi transaksional ini menjadi salah satu fokus utama dalam penelitian fundamental *Smart Engineering*. Entitas cerdas di dalam sistem ini dimodelkan menggunakan **Smart Engine Abstraction (SEA)**, yang secara konseptual dirancang untuk mentransformasikan berbagai input (baik berupa gaya maupun nilai) menjadi output yang berharga melalui mekanisme konversi transformasional maupun transaksional.

Tantangan utama riset di masa depan adalah **mengembangkan model matematika atau model formal untuk mengkuantifikasi dan mengoptimalkan tingkat konversi transaksional antar-dimensi PSKVE ini**. Sebagai contoh, para peneliti perlu merumuskan cara untuk menghitung seberapa besar dan efektif “energi pengetahuan” (seperti inovasi dari sebuah perangkat lunak baru) dapat dikonversi secara nyata menjadi “energi nilai” (seperti keuntungan finansial bagi perusahaan atau nilai manfaat bagi penggunaannya).

Smart Engine Abstraction (SEA)

Dalam kerangka *Smart Engineering*, **Smart Engine Abstraction (SEA)** adalah model konseptual inti yang digunakan untuk merancang dan menganalisis sistem yang mentransformasikan energi dan memiliki kecerdasan. Model SEA mengabstraksikan sebuah mesin sebagai entitas otonom dan dapat dikendalikan, yang secara spesifik berfungsi mengubah input (baik berupa gaya mekanis maupun nilai abstrak) menjadi output yang berharga. SEA dapat diterapkan lintas domain (baik perangkat keras, perangkat lunak, maupun perangkat spasial) dan tersusun atas komponen-komponen seperti *Source/Intake*, *Encoder*, *Internal Power*, *Decoder*, *Solution Value*, *Control*, dan *Flywheel*.

Agar desain SEA yang kompleks ini dapat dinalar, disimulasikan, dan dievaluasi oleh mesin komputasi (seperti pada simulasi berbasis Prolog), SEA harus dipetakan ke dalam struktur data formal menggunakan **Konsep Inti Ontologi**. Integrasi ini dilakukan dengan cara berikut:

- **Kelas (Classes) dan Individu (Individuals) sebagai Representasi Entitas SEA:** Di dalam basis pengetahuan ontologis (misalnya menggunakan bahasa Prolog), komponen-komponen sistem utama yang beroperasi sebagai SEA—seperti sistem kemudi otonom pada kendaraan, pusat manajemen lalu lintas, atau sistem manajemen jaringan energi—dideklarasikan secara eksplisit sebagai **Individu** (entitas spesifik) yang bernaung di bawah suatu **Kelas**. Representasi ini memastikan setiap mesin cerdas diakui eksistensinya secara logis oleh program.

4. Smart Engine Abstraction (SEA)

- **Properti (Properties) untuk Memodelkan Siklus PUDAL dan Transformasi PSKVE:** Kemampuan internal dari setiap SEA diterjemahkan menjadi **Properti** atau atribut logis. Dalam ontologi, properti ini digunakan untuk merinci bagaimana sebuah SEA menjalankan siklus cerdasnya (PUDAL) dan bagaimana ia melakukan transformasi internal terhadap energi multi-dimensi PSKVE (seperti mengonversi *Knowledge Energy* algoritmik menjadi efisiensi *Product Energy*). Komponen-komponen penyusun SEA (seperti *Encoder* atau *Flywheel*) juga dapat dimodelkan sebagai atribut turunan dari individu tersebut.
- **Relasi (Relationships) untuk Menyimulasikan Interaksi antar-SEA:** Sebuah sistem yang kompleks sering kali terdiri dari banyak SEA yang saling terkait (disebut sebagai *system-of-systems*). Ontologi menggunakan **Relasi** (berupa aturan-aturan atau *rules* logika) untuk memodelkan interaksi dinamis dan aliran energi atau nilai di antara berbagai SEA yang terhubung. Pemodelan relasi ini memungkinkan sistem komputasi untuk memahami perilaku *emergent* (kemunculan sifat baru) yang terjadi saat berbagai mesin cerdas bertransaksi dan saling memengaruhi satu sama lain.

Secara keseluruhan, dengan memetakan Smart Engine Abstraction (SEA) ke dalam konsep inti ontologi, para insinyur tidak hanya mendapatkan bahasa atau pola desain bersama yang mempermudah kolaborasi lintas disiplin ., tetapi mereka juga berhasil mengubah konsep arsitektur mesin cerdas ini menjadi bentuk struktur data yang bisa dinalar dan dioptimalkan secara otomatis oleh mesin komputasi (kecerdasan buatan).

Bagian 2: Metodologi Multi-Domain (PICOC)

Dalam kerangka *Smart Engineering*, penyelesaian masalah teknik modern yang sangat kompleks dan lintas disiplin didekonstruksi ke dalam empat domain utama, yakni domain Aplikasi, Sistem, Teknologi, dan Riset Fundamental. Untuk meneliti, mengevaluasi, dan melaporkan solusi pada keempat lapisan domain ini secara sistematis, kerangka kerja ini memanfaatkan **Metodologi PICOC**.

Secara mendasar, metodologi PICOC berfungsi sebagai struktur pelaporan yang mengevaluasi sebuah perlakuan (solusi) baru terhadap solusi lama untuk mengukur peningkatan yang dihasilkan dalam memahami atau memecahkan suatu masalah. Komponen metodologi ini terdiri dari:

- **P (Population):** Kelompok target, entitas, atau kumpulan data yang diteliti.
- **I (Intervention):** Perlakuan inovatif berupa solusi, sistem, teknologi, atau proses baru yang diusulkan.
- **C (Control):** Solusi, sistem, atau teknologi lama yang sudah ada dan digunakan sebagai garis dasar (*baseline*) pembandingan.
- **O (Outcome):** Efek yang dapat diukur, seperti peningkatan performa, nilai-nilai baru yang dihasilkan, atau temuan pengetahuan baru.
- **Cx (Context):** Masalah spesifik, batasan kebutuhan, atau kesenjangan pengetahuan yang menjadi latar belakang perancangan.

Keunikan metodologi lintas-domain ini dalam ekosistem *Smart Engineering* adalah **penerapan struktur PICOC yang diiterasi secara spesifik pada keempat lapisan (domain) rekayasa**:

- **Pada Domain Aplikasi (PICOC-A):** Fokus evaluasi berada pada penyelesaian masalah pemangku kepentingan (*stakeholders*). Sistem membandingkan solusi baru yang ditawarkan dengan praktik yang digunakan masyarakat saat ini, lalu mengukur hasil akhirnya dalam bentuk peningkatan efisiensi, pengurangan biaya, kepuasan pengguna, atau manfaat nyata (*Value Energy*).

- **Pada Domain Sistem (PICOC-S):** Berfokus pada pengujian rancangan sistem terintegrasi (contohnya perancangan kendaraan cerdas secara utuh) yang akan mewujudkan solusi pada lapisan aplikasi. Di sini, sistem baru dievaluasi performanya (seperti akurasi, kecepatan, pemanfaatan sumber daya, atau keandalan) melawan arsitektur sistem yang sudah ada sebelumnya.
- **Pada Domain Teknologi (PICOC-T):** Berfokus pada evaluasi mesin penggerak (*Smart Engine*) atau modul teknologi inti di dalam sistem. Metodologi ini mengukur seberapa efisien teknologi baru tersebut dalam melakukan tugas konversi spesifik (seperti konversi *Product Energy* menjadi *Service Energy*) jika dibandingkan dengan metode atau instrumen lama.
- **Pada Domain Riset Fundamental (PICOC-F):** Berada di lapisan terdalam untuk memvalidasi prinsip-prinsip saintifik, entitas ontologis, atau teori konversi energi PSKVE yang menjadi landasan inovasi. Metodologi membandingkan kerangka teori baru dengan model lama untuk mengukur penemuan prinsip pengetahuan baru (*Knowledge Energy*) yang tervalidasi secara logika.

Penerapan struktural metodologi PICOC lintas domain ini sangat krusial karena **memastikan adanya koherensi logis antar-lapisan desain**. Dengan pendekatan ini, para insinyur dapat dengan jelas mendemonstrasikan bagaimana keberhasilan penemuan teori di lapisan Fundamental akan menyokong efisiensi konversi pada lapisan Teknologi, yang kemudian menghidupkan kecerdasan operasional di lapisan Sistem, dan bermuara pada penyelesaian masalah yang berdampak langsung bagi manusia di lapisan Aplikasi.

5

Domain Aplikasi (A)

Dalam kerangka desain *Smart Engineering*, metodologi PICOC (*Population, Intervention, Control, Outcome, Context*) diterapkan secara berlapis pada empat domain hierarkis—yakni Aplikasi, Sistem, Teknologi, dan Riset Fundamental—untuk mengevaluasi dan menyusun laporan inovasi secara terstruktur.

Pada lapisan teratas, yaitu **Domain Aplikasi (A)**, metodologi ini berfokus secara eksklusif pada upaya **menemukan solusi yang lebih baik untuk memecahkan masalah nyata yang dihadapi oleh para pemangku kepentingan (stakeholders)**.

Penerapan kelima komponen PICOC pada Domain Aplikasi (PICOC-A) dijabarkan sebagai berikut:

- **P(A) - Populasi (Population):** Menargetkan subjek atau kelompok yang diteliti, yang dalam domain ini berupa **para pemangku kepentingan, seperti pengguna akhir, organisasi, atau masyarakat luas**.
- **I(A) - Intervensi (Intervention):** Merepresentasikan **solusi baru yang diusulkan** kepada masyarakat. Solusi pada tingkat aplikasi ini merupakan manifestasi langsung dari sistem cerdas yang telah dirancang pada Domain Sistem (S).
- **C(A) - Kontrol (Control):** Merupakan **solusi lama, sistem eksisting, atau praktik yang saat ini digunakan** oleh populasi. Analisis pada bagian ini mencakup evaluasi terhadap batasan, kelemahan, dan alasan mengapa solusi lama tersebut sudah tidak lagi memadai.

5. Domain Aplikasi (A)

- **O(A) - Hasil (Outcome):** Mengukur secara langsung **peningkatan performa atau manfaat yang dirasakan oleh para pemangku kepentingan**. Metrik pengukurannya sangat berkaitan dengan dimensi *Value Energy*, seperti tingkat efisiensi, pengurangan biaya finansial, kemudahan penggunaan (*usability*), hingga kepuasan pengguna.
- **Cx(A) - Konteks (Context):** Mendefinisikan **masalah spesifik yang melatarbelakangi perancangan**, beserta spesifikasi kebutuhan (*requirements*) yang harus dipenuhi agar solusi tersebut dapat dikatakan berhasil memecahkan masalah pemangku kepentingan.

Dalam metodologi multi-domain ini, keberhasilan di Domain Aplikasi (A) merupakan pembuktian tertinggi dari seluruh siklus inovasi rekayasa. Peningkatan performa yang diklaim di tingkat Aplikasi (O(A)) tidak berdiri sendiri, melainkan **harus didukung dan dibuktikan oleh keberhasilan arsitektur operasional pada tingkat Sistem (O(S)), yang pada gilirannya disokong oleh efisiensi mesin di tingkat Teknologi (O(T)) dan divalidasi oleh teori di tingkat Riset Fundamental (O(F))**. Struktur pelaporan berjenjang ini memastikan bahwa rancangan teknis yang rumit sekalipun akan selalu berorientasi pada penyelesaian masalah manusia di dunia nyata.

5.1. P: Stakeholder

Dalam metodologi pelaporan PICOC yang diterapkan pada Domain Aplikasi (PICOC-A), komponen **P (Populasi)** secara spesifik direpresentasikan sebagai **Stakeholder atau Pemangku Kepentingan**.

Dalam kerangka *Smart Engineering*, Domain Aplikasi berfokus pada upaya memecahkan masalah nyata, sehingga populasi yang diteliti bukanlah data atau mesin, melainkan subjek manusia atau entitas yang mengalami masalah tersebut.

Berikut adalah peran dan posisi **P: Stakeholder** dalam konteks Domain Aplikasi (A):

- **Definisi dan Wujud Stakeholder (P(A)):** Stakeholder dapat berupa **pengguna akhir (users), organisasi, institusi, masyarakat luas, hingga para insinyur itu sendiri**. Sebagai contoh kasus ekstrem, Perserikatan Bangsa-Bangsa (PBB) dapat bertindak sebagai stakeholder (P) dalam proyek pengiriman pesan ke galaksi lain. Dalam proyek rekayasa perangkat lunak, para insinyur yang membutuhkan alat desain otomatis juga merupakan wujud dari stakeholder. Jika pengujian aplikasi melibatkan

manusia secara langsung, definisi $P(A)$ ini harus mencakup rincian seperti kriteria seleksi, ukuran sampel, dan demografi partisipan.

- **Sebagai Penentu Titik Awal Masalah (Konteks):** Analisis di Domain Aplikasi selalu dimulai dengan mengidentifikasi masalah spesifik ($Cx(A)$) yang tengah dihadapi oleh para stakeholder ini. Analisis harus mendefinisikan dengan jelas apa kesenjangan (*gap*) antara kebutuhan atau keinginan stakeholder dengan solusi yang tersedia bagi mereka saat ini.
- **Target Evaluasi Dampak (Outcome):** Motivasi utama dari penciptaan solusi baru (Intervensi) selalu dinilai berdasarkan seberapa penting penyelesaian masalah tersebut bagi kelompok stakeholder ($P(A)$). Oleh karena itu, keberhasilan suatu sistem diukur dari **seberapa besar peningkatan performa ($O(A)$)—seperti pengurangan biaya, efisiensi waktu, atau kepuasan—yang dirasakan dan berdampak langsung pada para stakeholder tersebut**. Sebagai contoh, inovasi platform cerdas dinilai berhasil di tingkat aplikasi jika ia memberikan implikasi nyata berupa alat bantu yang lebih otomatis dan cerdas bagi insinyur sebagai stakeholder-nya.

Secara keseluruhan, pemosisian **Stakeholder sebagai Populasi** memastikan bahwa seluruh inovasi teknologi (T) dan sistem (S) yang dikembangkan oleh insinyur tidak kehilangan arah, dan selalu berorientasi pada penyelesaian masalah yang relevan serta memberikan manfaat (*Value Energy*) yang nyata bagi manusia.

5.2. I: Solusi Baru

Dalam metodologi pelaporan PICOC yang diterapkan pada Domain Aplikasi (PICOC-A), komponen **I (Intervention / Intervensi)** secara spesifik merepresentasikan **Solusi Baru (New Solution)** yang ditawarkan atau diusulkan untuk memecahkan masalah yang dihadapi oleh para pemangku kepentingan (*stakeholders*).

Berikut adalah peran, karakteristik, dan posisi **I: Solusi Baru** di dalam ekosistem Domain Aplikasi (A):

- **Puncak dari Lapisan Multi-Domain:** Di dalam kerangka kerja *Smart Engineering*, Solusi Baru di tingkat Aplikasi ($I(A)$) tidak berdiri sendiri sebagai ide yang terisolasi. Solusi ini merupakan **manifestasi akhir yang dibangun langsung berdasarkan arsitektur sistem yang telah dirancang pada Domain Sistem ($I(S)$)**. Lebih jauh lagi, wujud solusi ini dimungkinkan karena ia memanfaatkan kemajuan instrumen di

5. Domain Aplikasi (A)

lapisan Teknologi (I(T)) dan divalidasi oleh teori-teori pada lapisan Riset Fundamental (I(F)).

- **Fokus pada Peluang dan Pembaruan:** Menghadirkan I(A) berarti memperkenalkan sebuah peluang solusi yang benar-benar baru (*novel*) dan diklaim lebih baik. Penyusunan I(A) harus dirancang sedemikian rupa untuk menunjukkan dengan jelas bagaimana solusi ini memperluas, memperbaiki, atau menawarkan cara kerja yang sangat berbeda (*diverge*) dibandingkan dengan upaya-upaya pemecahan masalah di masa lalu.
- **Contoh Implementasi (Platform Terintegrasi):** Sebagai contoh konkret dalam literatur rekayasa, Solusi Baru (I(A)) yang diusulkan kepada insinyur (sebagai *stakeholder*) dapat berupa sebuah **Platform Terintegrasi Prolog-Python**. Platform ini diusulkan sebagai intervensi solusi baru untuk mengatasi kelemahan alur kerja rekayasa konvensional, dengan memberikan kemampuan penalaran simbolik sekaligus pemrosesan data algoritmik di dalam satu wadah.
- **Subjek Utama Evaluasi:** Pada tahap penyusunan metodologi, bentuk dari Solusi Baru (I(A)) harus dijabarkan dengan sangat rinci terkait implementasi atau pengujiannya. Hal ini karena I(A) akan langsung dikomparasikan atau “diadu” dengan Solusi Lama/Praktik Eksisting (*Control* / C(A)) untuk membuktikan seberapa besar peningkatan performa atau manfaat akhir (*Outcome* / O(A)) yang berhasil diciptakan bagi para pemangku kepentingan.

Secara keseluruhan, **I(A)** merupakan jawaban inovatif final—dapat berupa produk, layanan, atau sistem—yang diserahkan secara langsung kepada pemangku kepentingan guna menyelesaikan kesenjangan masalah secara lebih cerdas dan efisien.

5.3. Cx: Masalah Spesifik

Dalam metodologi pelaporan PICOC (*Population, Intervention, Control, Outcome, Context*) yang diterapkan pada Domain Aplikasi (A), komponen **Cx (Konteks)** secara spesifik merujuk pada rumusan masalah, batasan (*constraints*), lingkungan, serta spesifikasi kebutuhan (*requirements*) yang harus diselesaikan untuk para pemangku kepentingan (*stakeholders*).

Di dalam ekosistem perancangan *Smart Engineering*, **Cx: Masalah Spesifik** menduduki posisi fundamental dengan peran-peran berikut:

- **Sebagai Titik Tolak dan Pernyataan Masalah (Problem Statement):** Cx(A) merupakan fondasi utama yang memotivasi seluruh proses rekayasa. Pada tahap ini, insinyur harus mendefinisikan secara jelas apa yang menjadi masalah utama pemangku kepentingan (P(A)), serta **mengidentifikasi kesenjangan (gap) antara kondisi ideal yang mereka butuhkan dengan ketersediaan solusi pada saat ini.**
- **Mendefinisikan Kriteria Keberhasilan Solusi (Requirements):** Konteks tidak sekadar berisi deskripsi masalah, tetapi juga merumuskan prasyarat yang wajib dipenuhi. Parameter yang dirumuskan dalam Cx(A) ini akan menjadi kompas bagi insinyur untuk memastikan bahwa Solusi Baru (I(A)) dan arsitektur sistem (S) yang mereka rancang benar-benar relevan dan mampu menjawab persoalan pemangku kepentingan.
- **Memberikan Batasan dalam Metodologi Pengujian:** Ketika insinyur merancang pengaturan eksperimen di Domain Aplikasi, komponen Cx(A) dijabarkan kembali untuk **merinci latar atau pengaturan spesifik (setting) serta batasan dari pengujian aplikasi tersebut.** Ini memastikan bahwa solusi yang diuji berada pada lingkungan yang mencerminkan realitas permasalahan yang dihadapi pengguna.
- **Sebagai Tolok Ukur Interpretasi Hasil:** Dalam fase evaluasi akhir, peningkatan performa atau manfaat yang berhasil diciptakan (*Outcome / O(A)*) harus selalu diinterpretasikan ulang berdasarkan konteks masalah awalnya (Cx(A)). Keberhasilan inovasi rekayasa diukur dari **sejauh mana solusi tersebut secara langsung mampu menjawab pertanyaan-pertanyaan atau kesenjangan awal yang ditetapkan di dalam masalah spesifik ini.**

Secara keseluruhan, pemosisian **Cx (Konteks)** di dalam Domain Aplikasi bertindak sebagai “jangkar” yang memastikan bahwa kerumitan teknologi dan sistem di lapisan bawahnya tidak akan kehilangan arah. Cx(A) memastikan bahwa setiap tetes inovasi yang dilakukan selalu berorientasi pada pemecahan masalah dunia nyata bagi manusia.

6

Domain Sistem (S)

Dalam metodologi pelaporan PICOC (*Population, Intervention, Control, Outcome, Context*) yang diterapkan secara berlapis pada kerangka *Smart Engineering*, **Domain Sistem (S)** atau *System Layer* menduduki posisi krusial sebagai jembatan penghubung antara penyelesaian masalah pengguna di Domain Aplikasi (A) dan mesin penggerak inti di Domain Teknologi (T).

Fokus utama dari Domain Sistem (S) adalah **mencari atau merancang sebuah arsitektur sistem terintegrasi yang mampu memenuhi spesifikasi kebutuhan (dari domain aplikasi) sekaligus mewadahi dan menjalankan instrumen inovatif (dari domain teknologi)**.

Penerapan kelima komponen PICOC secara spesifik pada Domain Sistem (PICOC-S) dijabarkan sebagai berikut:

- **P(S) - Populasi (Population):** Berbeda dengan Domain Aplikasi yang menargetkan manusia (pemangku kepentingan), populasi pada Domain Sistem merujuk pada objek pengujian teknis. Ini mencakup **kumpulan data (datasets), lingkungan simulasi, model rekayasa, atau ranah pengujian (testbeds)** tempat sistem tersebut dioperasikan dan dievaluasi.
- **I(S) - Intervensi (Intervention):** Merepresentasikan **sistem baru yang diusulkan**, yang telah diintegrasikan dengan teknologi-teknologi kunci (*Key Technologies*). Dalam contoh transportasi, I(S) adalah rancangan kendaraan cerdas secara utuh. Sedangkan dalam ranah rekayasa perangkat lunak, I(S) dapat berupa arsitektur platform terintegrasi yang menggabungkan mesin Prolog (penalaran) dan Python (komputasi algoritma).

6. Domain Sistem (S)

- **C(S) - Kontrol (Control):** Merupakan sistem dasar (baseline) atau sistem eksisting (lama) yang digunakan sebagai pembanding. Pada perancangan platform perangkat lunak cerdas, C(S) adalah sistem Prolog mandiri atau skrip Python tradisional yang belum terintegrasi satu sama lain.
- **O(S) - Hasil (Outcome):** Mengukur efektivitas operasional dari sistem yang dirancang melalui metrik performa tingkat sistem. Metrik pengukurannya mencakup **tingkat akurasi, kecepatan (speed), pemanfaatan sumber daya (resource utilization), tingkat keandalan (reliability), hingga fleksibilitas alur kerja.**
- **Cx(S) - Konteks (Context):** Mendefinisikan **kebutuhan spesifik akan sebuah sistem yang lebih baik**, yang mampu memenuhi prasyarat solusi aplikasi sekaligus mewadahi prasyarat teknologi dasar di baliknya. Ini juga merinci latar belakang batasan desain dari sistem tersebut.

Posisi Domain Sistem (S) dalam Hierarki Lintas Domain Di dalam struktur pelaporan yang komprehensif, keberhasilan pada Domain Sistem tidak berdiri sendiri, melainkan memiliki interkoneksi logis yang kuat dengan domain di atas dan di bawahnya:

- Peningkatan performa di tingkat sistem (O(S)) berfungsi sebagai bukti teknis yang secara langsung **mendukung dan memungkinkan terwujudnya manfaat bagi pemangku kepentingan di tingkat Aplikasi (O(A)).** Sebaliknya, keberhasilan operasional sistem secara keseluruhan (O(S)) **dimungkinkan oleh dan sangat bergantung pada peningkatan performa dan efisiensi dari modul mesin/instrumen yang berada di lapisan Teknologi (O(T)).**

Contoh Metaphorikal dalam Rekayasa Dalam studi perancangan transportasi, Domain Sistem berfokus secara eksklusif pada perancangan **kendaraan sebagai sebuah entitas utuh**. Kendaraan tersebut dipandang sebagai sebuah sistem cerdas kompleks yang memanfaatkan siklus PUDAL untuk menavigasi lingkungan kerjanya, sambil terus-menerus mengelola dan mengoptimalkan Energi Produk (bahan bakar) agar dapat mengangkut beban penumpang secara aman dan efisien pada kecepatan yang diharapkan.

6.1. P: Testbeds/Dataset

Dalam metodologi pelaporan PICOC yang diterapkan pada Domain Sistem (PICOC-S), komponen **P (Populasi)** mengalami pergeseran makna yang signifikan dibandingkan domain di atasnya. Jika pada Domain Aplikasi (A) populasi merujuk pada manusia atau pemangku kepentingan (*stakeholders*), pada Domain Sistem (S), **Populasi direpresentasikan secara teknis**

sebagai testbeds (ranah pengujian), datasets (kumpulan data), test sets (set pengujian), lingkungan simulasi, model rekayasa, hingga system states (status/kondisi sistem).

Pemosisian **P: Testbeds/Dataset** dalam ekosistem Domain Sistem memiliki peran dan wawasan fundamental sebagai berikut:

- **Sebagai Lingkungan Validasi Terkontrol:** Domain Sistem (S) berfokus pada perancangan arsitektur sistem terintegrasi (seperti kendaraan cerdas utuh atau platform perangkat lunak) yang mewadahi teknologi inti. Sebelum sebuah sistem diserahkan kepada manusia (pemangku kepentingan) di Domain Aplikasi, sistem baru (Intervensi / I(S)) dan sistem lama (Kontrol / C(S)) harus diuji dan dikomparasikan di dalam lingkungan eksperimental yang presisi. *Testbeds*, *datasets*, dan lingkungan simulasi inilah yang bertindak sebagai “subjek uji” atau populasi tempat evaluasi operasional itu berlangsung.
- **Wujud Nyata dalam Rekayasa Perangkat Lunak:** Dalam contoh perancangan platform perangkat lunak terintegrasi seperti Prolog-Python, wujud dari populasi P(S) ini adalah model-model rekayasa, kumpulan dataset yang diolah, serta berbagai kondisi sistem (*system states*). Sistem baru diuji dengan cara memberikan berbagai skenario dari dataset tersebut untuk melihat seberapa tangguh arsitektur sistem dalam menangani tugas inferensi logis dan komputasi numerik secara bersamaan.
- **Fondasi bagi Pengukuran Performa (Outcome):** Penetapan spesifikasi *testbeds* atau *datasets* (P(S)) sangat krusial karena ia menjadi landasan tempat metrik keberhasilan sistem (O(S)) akan diukur. Keberhasilan sistem tidak diukur dari persepsi pengguna pada tahap ini, melainkan dari metrik objektif dan kuantitatif seperti akurasi pemrosesan dataset, kecepatan komputasi (*throughput* atau *latency*), pemanfaatan sumber daya, hingga tingkat keandalan simulasi. Penjelasan rincian tentang *testbeds* dan *datasets* di dalam metodologi memastikan bahwa pengujian sistem tersebut bersifat objektif dan dapat direproduksi (*reproducible*) oleh insinyur lain.

Singkatnya, di dalam kerangka kerja *Smart Engineering*, **P: Testbeds/Dataset** bertindak sebagai laboratorium pengujian. Evaluasi teknis yang sukses di atas *testbed* atau dataset simulasi ini akan menghasilkan bukti peningkatan performa sistem yang sah, yang pada gilirannya merupakan prasyarat mutlak sebelum solusi tersebut diklaim mampu memberikan manfaat nyata bagi manusia di dunia nyata.

6.2. I: Arsitektur Sistem Cerdas

Dalam metodologi pelaporan PICOC yang diterapkan pada Domain Sistem (PICOC-S), komponen **I (Intervensi)** secara spesifik merujuk pada **Arsitektur Sistem Cerdas (Proposed System)** yang diusulkan oleh insinyur.

Berikut adalah peran, karakteristik, dan posisi dari komponen **I: Arsitektur Sistem Cerdas** di dalam ekosistem Domain Sistem (S):

Definisi dan Fungsi Utama I(S) merepresentasikan sistem baru yang dirancang secara terintegrasi dengan menggabungkan berbagai teknologi kunci (*Key Technologies*). Dalam penjabaran metodologinya, I(S) harus merinci arsitektur sistem secara keseluruhan, komponen-komponen penyusunnya, algoritma yang digunakan, serta bagaimana teknologi dasar diintegrasikan menjadi satu kesatuan yang fungsional. Sistem cerdas ini dirancang khusus untuk memenuhi spesifikasi kebutuhan (*requirements*) yang telah ditetapkan demi menghadirkan solusi yang lebih baik di lapisan aplikasi.

Wujud I(S) dalam Rekayasa Perangkat Lunak Dalam studi kasus perancangan perangkat lunak *Smart Engineering*, wujud konkret dari I(S) adalah **Platform Terintegrasi Prolog-Python**. Arsitektur sistem cerdas ini dirancang dengan menggabungkan dua lingkungan yang berbeda:

- Mesin inferensi Prolog yang bertugas menangani penalaran simbolik, logika konseptual, dan manajemen pengetahuan.- Lingkungan *interpreter* Python beserta ekosistem *library*-nya yang bertugas menangani pemrosesan data numerik algoritmik, antarmuka pengguna, dan orkestrasi alur kerja.- Arsitektur ini disatukan oleh sebuah *bridge library* (seperti PySWIP atau Janus) yang memfasilitasi komunikasi antar-proses dan pertukaran data antara kedua bahasa tersebut.

Wujud I(S) dalam Metafora Transportasi Dalam contoh simulasi transportasi lintas domain, I(S) direpresentasikan oleh **kendaraan sebagai sebuah entitas cerdas yang utuh**, seperti halnya rancangan kendaraan otonom. Kendaraan (sistem) ini dirancang untuk secara aktif menjalankan siklus kecerdasannya (PUDAL) guna menavigasi lingkungan operasionalnya sembari mengelola energi produk (bahan bakar) yang dimilikinya secara efisien.

Posisi sebagai Jembatan Lintas Domain Di dalam hierarki pelaporan multi-domain, Arsitektur Sistem Cerdas (I(S)) menduduki posisi sentral yang menghubungkan lapisan bawah dan atasnya:

- Sistem terintegrasi ini (I(S)) baru bisa berfungsi dengan optimal jika didukung oleh penemuan atau efisiensi mesin cerdas/instrumen teknologi yang kuat di lapisan bawahnya, yaitu Intervensi Teknologi (I(T)).-

Sebaliknya, keberadaan arsitektur sistem operasional (I(S)) yang utuh ini merupakan prasyarat mutlak atau penyokong utama (*enabler*) agar Solusi Baru yang bermanfaat (I(A)) dapat benar-benar diwujudkan dan diserahkan kepada manusia (pemangku kepentingan) di lapisan Aplikasi.

7

Domain Teknologi (T)

Dalam kerangka kerja metodologi pelaporan PICOC multi-domain, **Domain Teknologi (T)** beroperasi pada lapisan krusial yang menjembatani antara penelitian prinsip-prinsip dasar di Domain Fundamental (F) dan integrasi operasional operasional di Domain Sistem (S).

Fokus utama dari Domain Teknologi adalah **menemukan, mengembangkan, atau memperbaiki Teknologi Kunci (Key Technology)**—yang umumnya berupa mesin atau modul khusus—agar mampu mengeksekusi tugas-tugas krusial, seperti komputasi yang kompleks atau konversi energi, secara jauh lebih efektif berdasarkan prinsip sains dan rekayasa.

Penerapan kelima komponen PICOC secara spesifik pada Domain Teknologi (PICOC-T) dijabarkan sebagai berikut:

- **P(T) - Populasi (Population):** Dalam domain ini, populasi yang menjadi subjek perlakuan atau pemrosesan bukanlah manusia atau sistem utuh, melainkan **energi sumber, nilai input, data mentah, atau material** yang akan diproses oleh mesin tersebut.
- **I(T) - Intervensi (Intervention):** Merepresentasikan **Mesin atau Modul Teknologi (Technological Engine/Module)** dengan **instrumen atau metode baru yang diusulkan**. Dalam paradigma *Smart Engineering*, intervensi teknologi ini kerap kali diabstraksikan sebagai *Smart Engine* yang mengadopsi siklus kecerdasan PUDAL untuk terus mengoptimalkan konversi energi di dalamnya.

7. Domain Teknologi (T)

- **C(T) - Kontrol (Control):** Merupakan **mesin, modul teknologi, instrumen, atau metode lama yang saat ini eksis**, yang digunakan sebagai garis dasar (*baseline*) pembandingan untuk menguji efisiensi inovasi baru.
- **O(T) - Hasil (Outcome):** Mengukur secara objektif **peningkatan performa di dalam menjalankan tugas konversi atau komputasi spesifik tersebut**. Metrik pada level ini meliputi efisiensi konversi yang lebih tinggi, peningkatan presisi, kecepatan pemrosesan (*faster processing*), hingga rasio sinyal terhadap gangguan (*signal-to-noise ratio*) yang lebih baik.
- **Cx(T) - Konteks (Context):** Mendefinisikan **tantangan spesifik teknis, sasaran, maupun batasan** dalam mencari instrumen yang tepat untuk menjalankan proses konversi energi/tugas komputasi dengan baik.

Posisi Domain Teknologi (T) dalam Hierarki Lintas Domain Di dalam struktur rekayasa yang komprehensif, keberhasilan pengujian pada Domain Teknologi (T) memiliki keterikatan logis yang tak terpisahkan dengan domain di atas dan di bawahnya:

- **Penyokong Utama Kinerja Sistem (S):** Hasil peningkatan performa pada mesin atau modul teknologi (O(T)) menjadi **prasyarat mutlak atau faktor pemungkin (enabler) bagi beroperasinya arsitektur sistem secara keseluruhan di Domain Sistem (O(S))**. Sebuah sistem cerdas hanya dapat memproses alur kerja dengan optimal apabila komponen mesin penggerakannya memiliki konversi energi yang berefisiensi tinggi.
- **Berlandaskan Lapisan Fundamental (F):** Inovasi instrumen mesin atau metode pada level ini (I(T)) tidak diciptakan dari ruang hampa, melainkan dibangun dan disokong oleh divalidasinya prinsip, penemuan efek baru, atau teori ilmiah yang diuji pada lapisan Riset Fundamental (O(F)).

Contoh Wujud Nyata dalam Rekayasa

- **Pada Metafora Transportasi:** Di tingkat teknologi, masalah diartikan sebagai cara menemukan atau mendesain teknologi konversi mesin (M) yang sanggup menghasilkan, menyimpan, dan mengubah energi sumber alamiah (E) menjadi energi kerja mekanik (U) untuk kendaraan tersebut. Contoh radikalnya adalah penemuan instrumen *Electromagnetic (EM) machine* yang diusulkan untuk mengonversi medan elektromagnetik antar-galaksi menjadi tenaga pendorong wahana antariksa.
- **Pada Rekayasa Perangkat Lunak Cerdas:** Dalam perancangan arsitektur terintegrasi, perwujudan dari intervensi teknologi (I(T)) adalah pengembangan **library jembatan atau bridge libraries** (seperti PySWIP atau Janus). Teknologi modul ini secara khusus memiliki tugas krusial

mengubah, mengelola pertukaran tipe data, dan memfasilitasi komunikasi komunikasi antar-proses yang berbeda, yakni antara mesin inferensi simbolik (Prolog) dengan komputasi numerik (Python). Peningkatan di ranah ini diukur ($O(T)$) dari seberapa kecil beban kinerja (*overhead*) komunikasi yang terjadi serta kemulusan aliran datanya.

7.1. P: Energi Sumber/Data Mentah

Dalam metodologi pelaporan PICOC yang diterapkan pada tingkat Domain Teknologi (PICOC-T), komponen **P (Populasi)** tidak merujuk pada manusia, pemangku kepentingan, ataupun sistem yang sudah utuh. Sebaliknya, komponen ini direpresentasikan secara spesifik sebagai **Energi Sumber (Source Energy)**, **Nilai Input, Data Mentah (Raw Data)**, **Fenomena**, atau **Material dasar**.

Berikut adalah peran dan posisi **P: Energi Sumber/Data Mentah** di dalam ekosistem perancangan Domain Teknologi (T):

- **Sebagai Objek yang Diproses atau Dikonversi:** Fokus utama dari Domain Teknologi adalah mencari, mengembangkan, atau memperbaiki Teknologi Kunci (*Key Technology*)—biasanya berupa mesin atau modul instrumen tertentu—agar dapat mengeksekusi tugas-tugas konversi atau komputasi yang krusial. Di dalam domain ini, **P(T)** adalah **bahan baku mentah yang akan dikenai tindakan atau diproses secara langsung oleh teknologi penggerak tersebut**.
- **Wujud dalam Metafora Transportasi (Fisik):** Jika merujuk pada desain mesin kendaraan, wujud dari populasi (P(T)) ini adalah **energi sumber (source energy / E) yang diperoleh atau ditangkap dari lingkungan (environment / W)**. Tantangan utamanya adalah bagaimana mesin tersebut mampu menyimpan dan mengubah energi sumber yang mentah ini menjadi energi kerja mekanis (*work energy / U*) secara maksimal untuk menggerakkan kendaraan.
- **Wujud dalam Rekayasa Perangkat Lunak (Informasional):** Dalam contoh perancangan platform terintegrasi seperti Prolog-Python, wujud P(T) bergeser dari entitas fisik menjadi data logis. Populasi di sini mengambil bentuk **kueri-kueri logika, struktur data, nilai-nilai input, basis pengetahuan, maupun tugas-tugas komputasi numerik mentah**. Teknologi atau modul yang dirancang (seperti pustaka *bridge* PySWIP atau Janus) bertugas memproses data-data mentah ini untuk memastikan konversi tipe data lintas bahasa pemrograman berjalan lancar tanpa hambatan komunikasi.

7. Domain Teknologi (T)

- **Faktor Penentu Evaluasi Efisiensi (Outcome):** Pendefinisian spesifikasi Energi Sumber atau Data Mentah (P(T)) sangat penting karena ia menjadi patokan awal untuk mengevaluasi performa teknologi (O(T)). Keberhasilan inovasi mesin atau algoritma di tingkat ini dinilai secara objektif dari **seberapa tinggi efisiensi konversinya, seberapa cepat waktu pemrosesannya, dan seberapa besar kepresisian yang dicapai saat teknologi tersebut mengolah kumpulan material atau data mentah ini.**

Singkatnya, pemosisian Energi Sumber atau Data Mentah sebagai “Populasi” di Domain Teknologi menegaskan bahwa pada lapisan rekayasa ini, fokus para insinyur bukan lagi pada pengguna akhir, melainkan murni **berkonsentrasi untuk mengoptimalkan bagaimana bahan baku paling dasar—baik berupa aliran energi fisik maupun lalu lintas data komputasional—dapat dikonversi secara efisien oleh instrumen mesin menjadi bentuk energi yang fungsional bagi sebuah sistem.**

7.2. I: Engine/Modul Teknologi

Dalam metodologi pelaporan PICOC yang diterapkan pada Domain Teknologi (PICOC-T), komponen **I (Intervensi)** secara spesifik merujuk pada **Mesin Teknologi (Technological Engine) atau Modul dengan instrumen maupun metode yang baru.**

Berikut adalah peran, karakteristik, dan posisi **I: Engine/Modul Teknologi** di dalam ekosistem Domain Teknologi (T):

- **Fungsi Utama sebagai Pengeksekusi Tugas Krusial:** Fokus dari I(T) adalah menghadirkan teknologi penentu (*Key Technology*) yang dirancang untuk mengeksekusi tugas-tugas inti secara efektif yang didasarkan pada prinsip-prinsip fundamental. Tugas-tugas ini umumnya mencakup proses konversi energi, komputasi yang sangat kompleks, atau eksekusi fungsi-fungsi spesifik lainnya. Pada saat merumuskan I(T), insinyur diwajibkan untuk mendeskripsikan secara rinci prinsip kerja dari mesin atau metode baru tersebut.
- **Wujud sebagai Smart Engine (Dalam Metafora Fisik):** Di dalam kerangka *Smart Engineering*, mesin teknologi ini kerap dikonseptualisasikan menggunakan model *Smart Engine Abstraction* (SEA). Sebagai sebuah *Smart Engine*, mesin (M) ini memanfaatkan siklus kecerdasan PUDAL untuk mengoptimalkan konversi energi—misalnya mengubah Energi Produk (bahan bakar) menjadi energi kerja mekanis (U)—sambil mengintegrasikan algoritma dan kecerdasan buatan (*Knowledge Energy*) di dalamnya. Sebuah

contoh fiktif namun radikal di literatur adalah usulan *Electromagnetic (EM) Machine* yang bertindak sebagai $I(T)$ untuk mengonversi tenaga medan elektromagnetik murni menjadi gaya dorong bagi wahana antariksa.

- **Wujud Modul Perangkat Lunak (Dalam Domain Informasional):** Apabila menilik pada perancangan platform perangkat lunak terintegrasi, wujud $I(T)$ bukanlah mesin fisik, melainkan teknologi peranti lunak inti. Dalam kasus integrasi Python dan Prolog, komponen intervensi ($I(T)$) direpresentasikan oleh mesin inferensi Prolog, *interpreter* Python, dan secara krusial **pustaka penghubung (bridge libraries) seperti PySWIP atau Janus**. Modul jembatan ini bekerja sebagai pengeksekusi konversi data mentah ($P(T)$), di mana modul ini bertugas menangani kerumitan konversi tipe data antara bahasa Python dan term Prolog, serta mengelola konvensi eksekusi fungsi secara lintas bahasa.
- **Fondasi Kinerja Sistem:** Engine atau modul teknologi ($I(T)$) memproses bahan baku berupa data mentah atau energi sumber ($P(T)$). Intervensi inovatif pada lapisan teknologi ini sangat esensial karena efisiensi, presisi, atau kecepatan (*Outcome* / $O(T)$) yang berhasil dicapai oleh mesin penukar energi atau modul kodenya ini akan menjadi tumpuan utama yang memungkinkan beroperasinya arsitektur sistem operasional ($I(S)$) secara utuh pada lapisan Domain Sistem.

Domain Riset Fundamental (F)

Dalam metodologi pelaporan PICOC lintas domain (*multi-domain*), **Domain Riset Fundamental (F)** menempati lapisan paling dasar dan esensial di dalam kerangka kerja desain rekayasa. Fokus utama dari Domain Fundamental bukanlah menciptakan produk akhir untuk pengguna, melainkan **menambah kumpulan pengetahuan (Body of Knowledge) tentang realitas, menemukan prinsip-prinsip sains atau rekayasa baru, dan memvalidasi teori-teori dasar.**

Penerapan kelima komponen PICOC secara spesifik pada Domain Riset Fundamental (PICOC-F) dijabarkan sebagai berikut:

- **P(F) - Populasi (Population):** Subjek yang diteliti pada level ini sangat abstrak, yakni berupa **fenomena alam atau konseptual, entitas-entitas fundamental, serta nilai atau energi sumber yang sedang diinvestigasi.**
- **I(F) - Intervensi (Intervention):** Merepresentasikan gagasan fundamental yang baru, yang dapat berupa **proses, teori, kerangka pemodelan (model), maupun pendekatan eksperimental baru** yang diusulkan oleh peneliti.
- **C(F) - Kontrol (Control):** Merupakan **teori, model, pendekatan, atau proses eksisting (lama)** yang digunakan sebagai landasan pembandingan untuk menguji keabsahan teori baru tersebut.
- **O(F) - Hasil (Outcome):** Keberhasilan di level ini tidak diukur dari uang atau kecepatan komputasi, melainkan dari **terciptanya pengetahuan baru**

8. Domain Riset Fundamental (F)

yang diinginkan, pemahaman mekanisme yang jauh lebih dalam, tervalidasinya prinsip-prinsip sains/logika, atau penemuan efek dan nilai yang sebelumnya belum diketahui.

- **Cx(F) - Konteks (Context):** Merupakan latar belakang akademis yang memotivasi riset, yakni kebutuhan akan pengetahuan baru, keinginan untuk memahami mekanisme fundamental, atau eksplorasi terhadap wilayah teoretis atau saintifik yang belum dipetakan, yang nantinya akan relevan untuk mendukung domain di lapisan atasnya.

Posisi Fundamental (F) dalam Hierarki Lintas Domain Di dalam struktur rekayasa yang komprehensif, Domain Riset Fundamental bertindak sebagai “akar” yang menyokong seluruh pohon inovasi. Keberhasilan dalam memvalidasi prinsip atau teori di lapisan Fundamental (O(F)) menjadi **fondasi absolut yang menyokong dan memungkinkan penciptaan instrumen mesin atau metode di Domain Teknologi (O(T))**. Tanpa adanya teori yang sah di level F, inovasi teknologi (T), arsitektur sistem (S), dan penyelesaian masalah di masyarakat (A) tidak akan memiliki pijakan keilmuan yang kuat.

Contoh Wujud Nyata dalam Rekayasa

- **Dalam Paradigma Smart Engineering & Konversi Energi:** Di tingkat fundamental, riset difokuskan pada upaya **memahami dan merekayasa prinsip konversi transaksional antara berbagai dimensi energi PSKVE**. Contoh yang lebih radikal pada metafora transportasi antargalaksi adalah penemuan prinsip dasar (F) tentang bagaimana mengonversi Energi Elektromagnetik menjadi kerja mekanis murni.
- **Dalam Rekayasa Perangkat Lunak Terintegrasi:** Pada pengembangan platform cerdas Prolog-Python, intervensi fundamental (I(F)) mencakup penerapan prinsip **logika formal (seperti klausa Horn di Prolog)**, **metodologi representasi pengetahuan (seperti desain ontologi)**, serta **teknik penalaran tingkat tinggi (seperti unifikasi dan backtracking)**. Pembuktian bahwa konsep ontologis (seperti Kelas, Individu, dan Properti) dapat dipetakan secara matematis ke dalam logika predikat merupakan keberhasilan di tingkat O(F) yang memungkinkan terbangunnya teknologi jembatan lintas-bahasa pemrograman.

8.1. P: Fenomena/Prinsip Sains

Dalam kerangka metodologi pelaporan PICOC lintas domain, **Domain Riset Fundamental (F)** adalah lapisan paling dasar yang murni berfokus pada perluasan kumpulan pengetahuan (*Body of Knowledge*) tentang realitas, penemuan prinsip baru, serta validasi teori. Pada lapisan abstrak

ini, komponen **P (Populasi)** tidak lagi merujuk pada pemangku kepentingan (manusia), sistem utuh, ataupun bahan baku fisik, melainkan direpresentasikan secara spesifik sebagai **Fenomena, Entitas Fundamental, atau Energi/Nilai Sumber yang sedang diinvestigasi.**

Peran dan posisi **P: Fenomena/Entitas Fundamental (P(F))** di dalam ekosistem riset fundamental dijabarkan sebagai berikut:

- **Sebagai Subjek Utama Investigasi Sains dan Logika:** Populasi di tingkat ini merupakan entitas yang menjadi fokus pengamatan teoretis. Bentuk dari P(F) bisa mencakup fenomena alam yang abstrak atau entitas mendasar dari sebuah sistem nilai. Dalam konteks ilmu komputer seperti pengembangan sistem rekayasa perangkat lunak terintegrasi (misalnya ontologi Prolog), wujud populasi (P(F)) ini mengambil bentuk **proposisi logis, konsep-konsep abstrak, relasi antar-entitas, hingga titik data fundamental.**
- **Target Evaluasi dari Teori atau Model Baru:** Tantangan utama (*Context* / Cx(F)) di domain ini adalah menjawab pertanyaan mendasar tentang mekanisme dari populasi (P(F)) tersebut karena adanya celah pengetahuan (*knowledge gap*). Untuk menjelaskan fenomena tersebut, peneliti mengusulkan sebuah Intervensi (I(F)) yang dapat berupa teori, kerangka pemodelan, atau pendekatan eksperimental yang sama sekali baru.
- **Contoh Eksplorasi pada Fenomena Energi:** Dalam metafora inovasi kendaraan lintas galaksi, entitas yang menjadi “populasi” penelitian (P(F)) adalah **fenomena medan energi elektromagnetik murni.** Riset fundamental akan meneliti fenomena ini untuk menemukan keahlian (*know-how*) dan prinsip sains baru tentang bagaimana mengonversi energi elektromagnetik teoretis tersebut menjadi sebuah gaya kerja mekanis yang nyata.
- **Batu Loncatan bagi Seluruh Hierarki Rekayasa:** Keberhasilan riset pada tingkat ini diukur dari seberapa dalam pemahaman baru yang didapat atau tervalidasinya prinsip-prinsip yang mengatur fenomena tersebut (*Outcome* / O(F)). Evaluasi yang sukses pada fenomena dasar (P(F)) ini akan menciptakan teori yang kokoh, yang mana teori ini **menjadi penyokong dan fondasi mutlak yang memungkinkan lahirnya instrumen di tingkat Teknologi (T), arsitektur operasional di tingkat Sistem (S), dan solusi inovatif di tingkat Aplikasi (A).**

Singkatnya, pemosisian **Fenomena atau Entitas Fundamental** sebagai **“Populasi”** menegaskan bahwa tujuan utama Domain Fundamental adalah menginterogasi realitas alam, hukum-hukum konversi energi, atau struktur

8. Domain Riset Fundamental (F)

logika dasar. Penyelidikan mendalam terhadap “populasi” abstrak inilah yang akan menghasilkan kebenaran saintifik untuk menopang seluruh piramida inovasi di dalam *Smart Engineering*.

8.2. I: Teori/Model Baru

Dalam kerangka kerja pelaporan multi-domain PICOC, komponen **I (Intervensi)** pada Domain Riset Fundamental (PICOC-F) secara spesifik direpresentasikan sebagai **Teori, Model, Proses, atau Pendekatan Eksperimental Baru**.

Berikut adalah peran, posisi, dan wujud **I: Teori/Model Baru (I(F))** di dalam ekosistem Domain Riset Fundamental:

1. Sebagai Jawaban atas Kesenjangan Pengetahuan (Knowledge Gap)
Domain Fundamental tidak ditujukan untuk merancang produk komersial, melainkan didorong oleh kebutuhan untuk memahami mekanisme dasar alam atau realitas yang belum dipetakan ($C_x(F)$). Oleh karena itu, Teori atau Model Baru ($I(F)$) yang diusulkan oleh peneliti berfungsi sebagai kerangka pikir atau hipotesis yang diuji untuk memvalidasi entitas fundamental atau fenomena yang sedang diteliti ($P(F)$).

2. Fondasi Absolut bagi Seluruh Lapisan Rekayasa
Dalam struktur hierarkis *Smart Engineering*, Teori atau Model Baru ($I(F)$) bertindak sebagai “akar” pemikiran. Keberhasilan dalam memvalidasi prinsip atau model baru ini menjadi pengetahuan atau pemahaman baru ($O(F)$) yang secara langsung **menyokong dan memungkinkan penciptaan instrumen mesin di lapisan Teknologi ($O(T)$), arsitektur operasional di lapisan Sistem ($O(S)$), dan pada akhirnya menghasilkan solusi nyata di lapisan Aplikasi ($O(A)$)**. Tanpa adanya teori yang sah ($I(F)$), keseluruhan inovasi di lapisan atasnya tidak akan memiliki landasan sains atau logika yang kuat.

3. Wujud $I(F)$ dalam Rekayasa Perangkat Lunak dan Kecerdasan Buatan
Dalam konteks pengembangan platform *Smart Engineering* yang terintegrasi, wujud nyata dari intervensi fundamental ($I(F)$) ini meliputi penerapan prinsip-prinsip sains komputer dasar, seperti:

- **Penerapan logika formal**, misalnya penggunaan *Horn clauses* (klausa Horn) yang menjadi basis dari bahasa Prolog.
- **Metodologi representasi pengetahuan**, seperti pemodelan desain Ontologi yang memanfaatkan Kelas, Individu, Properti, dan Relasi. Model teoretis ini dirancang untuk memastikan bahwa representasi konseptual manusia atas suatu domain teknik dapat diterjemahkan menjadi struktur

data yang dapat dinalar secara presisi oleh mesin (*machine-understandable model*).

4. Wujud I(F) dalam Metafora Fisik dan Konversi Energi Jika merujuk pada metafora desain transportasi dan energi, Teori atau Model Baru (I(F)) mempresentasikan penemuan prinsip-prinsip sains (*know-how*) terkait cara memanipulasi energi.

- Contoh teoretis ekstrem yang disebutkan dalam sumber adalah riset fundamental yang menemukan model cara **mengonversi tenaga Medan Elektromagnetik (EM) antar-galaksi menjadi gaya kerja mekanis murni** bagi sebuah wahana antariksa.- Dalam konteks energi yang lebih luas di kerangka *Smart Engineering*, I(F) berwujud pencarian dan pemodelan prinsip-prinsip **Konversi Transaksional (Transactional Conversion)**, yakni teori yang menjelaskan bagaimana dimensi energi PSKVE (seperti Energi Pengetahuan atau Energi Lingkungan) dapat dipertukarkan satu sama lain secara optimal.

Singkatnya, **I: Teori/Model Baru (I(F))** adalah pijakan awal dari keseluruhan siklus inovasi rekayasa. Ia adalah gagasan teoretis murni yang, apabila tervalidasi kebenarannya, akan bertransformasi menjadi hukum sains atau pola desain yang memandu para insinyur dalam membangun teknologi (T) dan sistem (S) yang lebih cerdas.

Bagian 3: Integrasi Teknologi

Dalam kerangka *Smart Engineering*, **integrasi teknologi** bukan sekadar upaya teknis untuk menggabungkan dua perangkat lunak, melainkan merupakan pergeseran paradigma untuk memadukan sistem kecerdasan alami dan kecerdasan buatan ke dalam seluruh siklus rekayasa. Tujuan utamanya adalah untuk menjembatani kesenjangan antara model konseptual (cara manusia memahami masalah) dengan data atau logika yang dapat diinterpretasikan dan diproses oleh mesin.

Domain Sistem (S) sebagai Pusat Integrasi Di dalam metodologi pelaporan multi-domain PICOC, proses integrasi ini berpusat pada **Domain Sistem (S)**. Di lapisan inilah sebuah Arsitektur Sistem Cerdas (direpresentasikan sebagai Intervensi Sistem / I(S)) dirancang secara spesifik untuk mewadahi dan menggabungkan berbagai “**Teknologi Kunci**” (**Key Technologies**). Sistem terintegrasi ini dirancang agar mampu memproses alur kerja dari komputasi matematis hingga penalaran logika secara bersamaan.

Wujud Utama Integrasi: Platform Prolog-Python Sumber-sumber yang diberikan menyoroti rancangan **Platform Terintegrasi Prolog-Python** sebagai contoh utama dan paling radikal dari integrasi teknologi di dalam *Smart Engineering*. Integrasi ini menyatukan dua kekuatan paradigma pemrograman yang sangat berbeda namun saling melengkapi:

- **Python (Kekuatan Algoritmik):** Berperan sebagai pengelola utama aplikasi yang menangani interaksi pengguna (UI), pemrosesan data numerik, pembelajaran mesin, dan orkestrasi alur kerja secara keseluruhan.
- **Prolog (Kekuatan Penalaran Simbolik):** Didelegasikan sebagai mesin penalaran khusus (*reasoning engine*) di latar belakang. Prolog menggunakan logika formal (seperti ontologi) untuk mengeksekusi inferensi logis, menjaga kepatuhan terhadap aturan sistem, dan menemukan kesimpulan tersembunyi dari relasi data yang kompleks.

Peran Modul Penghubung (Bridge Libraries) di Domain Teknologi (T) Agar sistem gabungan ini dapat beroperasi secara mulus, integrasi membutuhkan intervensi spesifik di lapisan bawahnya, yakni **Domain Teknologi (T)**. Wujud intervensi teknologi (I(T)) ini adalah penggunaan pustaka jembatan komunikasi (*bridge libraries*) seperti **PySWIP** atau **Janus**.

Teknologi kunci ini sangat vital karena ia bertugas menangani kerumitan komunikasi antar-proses dan **konversi tipe data (data marshalling) secara mulus**. Sebagai contoh, pustaka ini secara otomatis mengubah struktur *list* atau *string* dari lingkungan Python menjadi *term* atau *atom* yang dapat dipahami oleh Prolog, lalu mengubah kembali hasil kueri logis dari Prolog ke dalam bentuk *dictionary* di Python.

Manfaat dan Dampak Keberhasilan Integrasi (Outcomes) Keberhasilan integrasi teknologi ini memberikan manfaat sistemik (*System Outcome / O(S)*) yang sangat besar bagi praktik *Smart Engineering*, di antaranya:

- **Modularitas Arsitektur:** Integrasi ini menciptakan arsitektur sistem yang fleksibel, di mana bahasa komputasi terbaik digunakan secara eksklusif untuk tugas yang paling tepat. Logika aturan tetap terisolasi dengan rapi di Prolog, sementara pengolahan data murni diurus oleh Python.
- **Simulasi Dinamis dan Interaktif:** Integrasi ini memungkinkan Python untuk memanipulasi basis pengetahuan Prolog secara dinamis saat sistem berjalan (*runtime*), misalnya dengan menambahkan fakta baru (**assertz**) atau menghapusnya (**retract**) berdasarkan interaksi pengguna. Kemampuan ini sangat penting untuk membangun sistem yang sadar-konteks (*knowledge-aware*) yang mampu beradaptasi terhadap perubahan kondisi di dunia nyata.

Secara keseluruhan, integrasi teknologi dalam *Smart Engineering* memberdayakan para insinyur untuk tidak hanya melakukan perhitungan matematis, tetapi juga menyematkan “pengetahuan dan logika” eksplisit ke dalam desain rekayasa. Hasil akhirnya adalah penyelesaian masalah pemangku kepentingan (Domain Aplikasi) yang jauh lebih cerdas, otomatis, dan dapat diverifikasi jalan penalarannya.

9

Prolog (Penalaran Simbolik)

Prolog (*Programming in Logic*) adalah bahasa pemrograman tingkat tinggi yang dibangun di atas fondasi logika formal, secara spesifik menggunakan subset kalkulus predikat orde pertama yang dikenal sebagai klausa Horn. Dalam ranah *Smart Engineering*, **Prolog diandalkan sebagai mesin penalaran simbolik yang beroperasi secara deklaratif**, di mana insinyur cukup mendefinisikan *apa* kebenaran yang ada melalui “fakta” dan *apa* hubungan bersyarat di antaranya melalui “aturan”, tanpa perlu menyusun langkah-langkah prosedural tentang *bagaimana* cara menghitungnya.

Kebutuhan akan Integrasi Teknologi Meskipun sangat kuat dalam penalaran logis, Prolog memiliki keterbatasan jika digunakan secara mandiri untuk rekayasa modern. Alat rekayasa tradisional (seperti skrip Python standar atau basis data relasional) sangat unggul dalam komputasi numerik, pemrosesan data dalam jumlah masif, dan pembangunan antarmuka, tetapi mereka sering kali kewalahan saat harus menangani manipulasi simbolik, deduksi logis yang kompleks, atau kepatuhan terhadap aturan-aturan domain (*constraints*). Oleh karena itu, diperlukan **Integrasi Teknologi** yang menggabungkan kehebatan algoritmik dari ekosistem seperti Python dengan kemampuan penalaran simbolik dan representasi pengetahuan ontologis dari Prolog.

Wujud Integrasi pada Domain Teknologi (T) Di dalam lapisan Teknologi (I(T)), integrasi dua paradigma yang berbeda ini dimungkinkan oleh inovasi berupa **pustaka penghubung (bridge libraries) seperti PySWIP atau Janus**. Modul teknologi ini bertugas melakukan “penerjemahan” tingkat rendah yang krusial, seperti mengonversi atom dan struktur list di Prolog menjadi *string* dan struktur data bawaan di Python, dan begitu pula sebaliknya. Pustaka inilah

9. Prolog (*Penalaran Simbolik*)

yang memastikan komunikasi antar-proses dan pertukaran data antara domain komputasi numerik dan domain penalaran simbolik berjalan tanpa hambatan dan dengan latensi yang sangat minim.

Alur Kerja Dinamis pada Domain Sistem (S) Pada tingkat arsitektur sistem terintegrasi (I(S)), kolaborasi Prolog dan Python menghasilkan alur kerja simulasi atau aplikasi yang sangat cerdas dan dinamis:

- **Orkestrasi dan Manajemen State:** Python bertindak sebagai orkestrator yang menginisialisasi mesin Prolog, memuat basis pengetahuan dasar, serta menangani antarmuka pengguna dan komputasi matematis.
- **Penyuntikan Fakta Dinamis (*Dynamic Assertion*):** Selama aplikasi atau simulasi berjalan, Python dapat terus-menerus memperbarui kondisi dunia nyata dengan menyuntikkan (*meng-assert*) fakta-fakta baru secara dinamis ke dalam basis pengetahuan Prolog, atau menariknya kembali (*retract*) saat statusnya berubah.
- **Delegasi Penalaran:** Saat sistem perlu mengevaluasi aturan yang rumit—misalnya memastikan apakah desain struktur telah mematuhi aturan standar tertentu, atau apakah komponen sistem memenuhi syarat kelayakan logis—Python akan mengirimkan kueri ke Prolog. Mesin inferensi Prolog (menggunakan metode resolusi dan *backtracking*) akan mencari kebenaran logis dari kueri tersebut dan mengembalikan jawabannya kepada Python untuk divisualisasikan atau diproses lebih lanjut.

Dampak Integrasi (*Outcome / O(S) & O(T)*) Hasil dari integrasi teknologi ini adalah **arsitektur perangkat lunak yang sangat modular di mana setiap bahasa digunakan untuk tugas paling optimalnya**. Prolog difokuskan murni untuk mengorkestrasi logika kompleks dan pencarian pemenuhan kendala, sementara Python mengelola pemrosesan numerik dan interaksi sistem. Pada akhirnya, penyatuan penalaran simbolik ini dengan komputasi modern membuka jalan untuk menciptakan perangkat lunak *Smart Engineering* yang tidak hanya mampu “menghitung”, tetapi juga mampu “memahami” model konseptual dari sistem yang dirancang.

9.1. Representasi Fakta & Aturan

Dalam konteks penalaran simbolik menggunakan Prolog, **Representasi Fakta dan Aturan** merupakan fondasi utama untuk membangun basis pengetahuan (*knowledge base*) yang bersandar pada logika formal (khususnya subset kalkulus predikat yang dikenal sebagai klausa Horn). Berbeda dengan bahasa pemrograman algoritmik yang prosedural, Prolog beroperasi secara deklaratif:

insinyur hanya perlu mendefinisikan *apa* yang secara logis benar dan *bagaimana* hubungannya, tanpa perlu memprogram langkah komputasinya.

Representasi Fakta (Facts) Fakta adalah **pernyataan deklaratif yang menegaskan kebenaran tanpa syarat (kebenaran absolut) tentang objek di dalam domain masalah**. Dalam pemodelan ontologi rekayasa cerdas, fakta digunakan untuk mendeklarasikan entitas-entitas dasar beserta atributnya:

- **Kelas dan Individu:** Diwakili oleh predikat *unary* (satu argumen). Contohnya, `student(alice)`. secara eksplisit menyatakan sebuah fakta bahwa ‘alice’ (individu) adalah bagian dari kelas ‘mahasiswa’.
- **Properti dan Atribut:** Dinyatakan melalui predikat biner atau terner. Misalnya, fakta `has_age(alice, 20)`. menegaskan nilai atribut umur bagi individu tersebut.
- **Relasi Dasar:** Digunakan untuk menyatakan tautan langsung antar-entitas, seperti `prerequisite(cs202, cs101)`. yang berarti mata kuliah CS101 adalah prasyarat bagi CS202.

Representasi Aturan (Rules) Sementara fakta memberikan data mentah, aturan berfungsi untuk **mendefinisikan kebenaran kondisional dan membangun hubungan deduktif di antara fakta-fakta tersebut**. Aturan ditulis dalam format `Head :- Body.`, yang secara logis diinterpretasikan sebagai “Head bernilai benar *jika* semua syarat di dalam Body bernilai benar”.Fungsionalitas aturan ini sangat luas:

- **Menemukan Relasi Implisit:** Aturan memungkinkan sistem mendeduksi hubungan yang tidak dinyatakan secara tertulis. Sebagai contoh, `friends(X,Y) :- likes(X,Y), likes(Y,X)`. akan menyimpulkan bahwa X dan Y adalah teman jika mereka terbukti memiliki fakta saling menyukai.
- **Mewujudkan Hierarki dan Pewarisan (Inheritance):** Aturan digunakan untuk memetakan hierarki kelas. Aturan `mammal(X) :- cat(X)`. menetapkan logika bahwa siapa pun (X) yang berstatus sebagai kucing, secara otomatis mewarisi identitas sebagai mamalia.
- **Mengevaluasi Batasan dan Syarat Logis:** Dalam kasus sistem terintegrasi, aturan digunakan untuk penalaran kompleks. Contohnya, aturan `can_enroll(Student, Course)` pada simulasi sistem universitas tidak sekadar mengecek fakta pendaftaran, tetapi secara dinamis memverifikasi apakah mahasiswa eksis, apakah ia belum mengambil kelas tersebut, dan mendeduksi apakah ia memenuhi daftar prasyaratnya berdasarkan fakta yang ada.

9. Prolog (Penalaran Simbolik)

Sinergi dalam Skenario Integrasi Teknologi Penggabungan antara basis fakta yang solid dengan logika aturan inilah yang menjadi bahan bakar bagi kecerdasan penalaran Prolog. Saat diintegrasikan dengan Python, Python dapat menyuntikkan (meng-*assert*) fakta-fakta baru ke dalam Prolog yang merepresentasikan kondisi dunia nyata saat ini. Selanjutnya, saat sistem dihadapkan pada pertanyaan yang rumit, **Prolog akan secara otomatis menggunakan mesin inferensinya (melalui metode unification dan backtracking) untuk menelusuri fakta, mengevaluasi aturan secara rekursif, dan memberikan kesimpulan logis yang valid.** Pada akhirnya, arsitektur yang digerakkan oleh relasi simbolik ini melahirkan platform rekayasa yang sanggup “memahami” model desain secara konseptual.

9.2. Mesin Inferensi Logika

Di dalam bahasa pemrograman Prolog, **Mesin Inferensi Logika (Inference Engine)** adalah **komponen penggerak utama yang memungkinkan sistem untuk melakukan penalaran simbolik secara otomatis.** Berbeda dengan bahasa pemrograman algoritmik tradisional yang mengharuskan insinyur memprogram “bagaimana” (*how*) cara menghitung suatu masalah secara prosedural, Prolog beroperasi secara deklaratif dengan mengandalkan mesin inferensi bawaan ini untuk mendeduksi jawaban dari “apa” (*what*) yang sudah dideklarasikan dalam bentuk fakta dan aturan.

Mekanisme Kerja Mesin Inferensi Mesin inferensi Prolog dirancang untuk memproses logika formal, khususnya subset dari kalkulus predikat orde pertama yang dikenal sebagai klausa Horn. Dalam menjalankan evaluasi logis, mesin ini menggunakan beberapa mekanisme inti:

- **Pembuktian Teorema Resolusi (Resolution Theorem Prover):** Saat pengguna atau sistem mengajukan kueri (pertanyaan), mesin inferensi berfungsi sebagai pembukti teorema otomatis yang mencari tahu apakah pernyataan tersebut benar berdasarkan basis pengetahuan yang tersedia.
- **Pencarian Mendalam dengan Runut-Balik (Depth-First Search with Backtracking):** Untuk menjawab kueri, mesin mencari solusi dengan menelusuri fakta dan aturan secara mendalam (*depth-first*). Jika sebuah jalur evaluasi menemui jalan buntu (kondisi tidak terpenuhi), mesin akan secara otomatis melakukan *backtracking* (runut-balik) untuk mengeksplorasi cabang alternatif atau kombinasi variabel lain hingga menemukan solusi logis yang valid.

- **Unifikasi (Unification):** Mekanisme ini merupakan fondasi di mana mesin secara otomatis mencocokkan pola variabel dalam kueri dengan struktur fakta atau aturan yang ada di dalam ontologi.

Beroperasi dengan Asumsi Dunia Tertutup (Closed World Assumption) Karakteristik fundamental dari mesin inferensi Prolog adalah ia beroperasi di bawah prinsip *Closed World Assumption* (CWA). Prinsip ini menetapkan bahwa **jika mesin inferensi tidak dapat membuktikan suatu pernyataan bernilai benar dari fakta dan aturan yang ada, maka pernyataan tersebut diasumsikan mutlak salah** (juga dikenal sebagai *Negation as Failure*). Walaupun sangat efisien untuk batas domain yang pasti, sifat CWA ini menjadi tantangan tersendiri ketika Prolog digunakan untuk merancang sistem yang menghadapi informasi yang tidak lengkap dari dunia nyata.

Peran Sentral dalam Integrasi Sistem Rekayasa (Smart Engineering) Di dalam perancangan platform *Smart Engineering* modern, kekuatan mesin inferensi ini diisolasi untuk fokus pada hal yang paling ia kuasai, lalu diintegrasikan dengan bahasa multiguna seperti Python.

Dalam arsitektur terintegrasi tersebut:

- Python mendelegasikan tugas-tugas yang membutuhkan penyelesaian kendala (*constraint satisfaction*), deduksi yang rumit, pencarian rute berbasis aturan, atau pengecekan kepatuhan desain **secara eksklusif kepada mesin inferensi Prolog**.- Karena mesin inferensi ini bersifat *built-in* (bawaan), insinyur perangkat lunak tidak perlu lagi memprogram ulang logika pencarian atau deduksi kompleks dari nol. Begitu ontologi dan aturan didefinisikan, kemampuan penalaran tingkat tinggi sudah secara inheren tersedia untuk menjawab kueri rumit yang dikirimkan oleh Python.

Singkatnya, **Mesin Inferensi Logika adalah “otak” deduktif di balik Prolog**. Mesin inilah yang mengubah kumpulan fakta mentah dan aturan hierarkis menjadi sebuah sistem dinamis yang sanggup menalar, mencari kesimpulan tersembunyi, dan memastikan konsistensi konseptual dari sebuah sistem rekayasa.

9.3. Pemetaan Komponen Ontologi

Dalam ranah penalaran simbolik menggunakan Prolog, **Pemetaan Komponen Ontologi (Ontology Component Mapping)** adalah proses sistematis menerjemahkan model konseptual dari suatu domain (seperti kelas, atribut, dan relasi) ke dalam struktur logika formal Prolog, yakni dalam wujud fakta dan aturan. Pemetaan ini bertindak sebagai jembatan

9. Prolog (Penalaran Simbolik)

krusial yang mengubah pemahaman konseptual manusia menjadi representasi data logis yang dapat dipahami dan dinalar secara otomatis oleh mesin komputasi.

Secara spesifik, komponen-komponen inti ontologi dipetakan ke dalam konstruksi bahasa Prolog melalui pendekatan berikut:

- **Pemetaan Kelas (Classes) dan Individu (Individuals):** Kelas direpresentasikan menggunakan predikat *unary* (satu argumen), di mana nama predikat merepresentasikan nama kelas, dan argumennya adalah individu di dalamnya. Individu itu sendiri ditulis sebagai *atom* atau konstanta. Sebagai contoh, pernyataan `student(alice).` secara langsung memetakan individu berwujud atom 'alice' sebagai anggota dari kelas 'student'. Relasi keanggotaan kelas yang eksplisit juga dapat dituliskan dengan predikat biner, seperti `isa(louis, man)..`
- **Pemetaan Properti dan Atribut (Properties/Attributes):** Atribut fisik maupun abstrak yang dimiliki oleh suatu entitas dipetakan menjadi fakta Prolog, umumnya menggunakan predikat biner atau terner. Struktur standarnya adalah `attribute(ObjectID, AttributeName, Value)`, contohnya `property(human, legs, two).` atau pemetaan spesifik individu seperti `has_age(alice, 20)..` Keunggulan pemetaan ini di Prolog adalah **nilai properti tidak hanya terbatas pada deklarasi fakta mutlak, tetapi juga bisa diturunkan secara dinamis melalui aturan (deduksi) logis.**
- **Pemetaan Relasi (Relationships):** Keterkaitan makna antar-individu atau antar-kelas diekspresikan secara alami menggunakan predikat biner atau *N-ary*. Relasi bisa berwujud koneksi langsung (misalnya `father_of(joe, paul).`), atau direpresentasikan sebagai **relasi kompleks turunan (derived relationships) yang dirumuskan melalui logika aturan.** Contohnya adalah aturan `friends(X,Y) :- likes(X,Y), likes(Y,X).` yang memetakan bahwa sistem dapat secara mandiri menyimpulkan relasi pertemanan antara X dan Y apabila keduanya terbukti memiliki fakta saling menyukai.
- **Pemetaan Hierarki Kelas dan Pewarisan (Class Hierarchies & Inheritance):** Hierarki ontologi sangat efektif dipetakan menggunakan aturan (*rules*) Prolog. Hubungan *subclass* (atau *a_kind_of*) didefinisikan dengan aturan seperti `mammal(X) :- cat(X).`, yang menegaskan bahwa secara logis setiap instans dari kelas kucing juga mewarisi identitas kelas mamalia. Lebih lanjut, **pemetaan aturan hierarkis ini secara otomatis mengaktifkan mekanisme pewarisan sifat (property inheritance).** Properti yang dimiliki oleh superkelas akan dieksplorasi secara rekursif sehingga diwariskan ke subkelas di bawahnya. Prolog juga memiliki

kemampuan mendefinisikan aturan pengecualian (*property overriding*), di mana sistem akan memprioritaskan pengecekan nilai atribut lokal yang spesifik terlebih dahulu sebelum menggunakan nilai bawaan yang diwariskan.

Signifikansi dalam Konteks yang Lebih Luas (Smart Engineering)

Di dalam kerangka kerja multi-domain *Smart Engineering*, proses pemetaan ontologi ke konstruksi Prolog menduduki posisi sentral pada Domain Riset Fundamental (I(F)).

Dengan memetakan komponen ontologi menjadi sekumpulan klausa Horn (fakta dan aturan logis), para insinyur tidak sekadar membuat pangkalan data statis, melainkan **menciptakan model pengetahuan deklaratif yang “hidup”**. Pemetaan langsung ini memastikan bahwa mesin inferensi Prolog (*built-in inference engine*) dapat menggunakan algoritma unifikasi dan penelusuran runut-balik (*backtracking*) untuk mengeksekusi penalaran simbolik tingkat tinggi. Pada akhirnya, hal ini memungkinkan sistem cerdas untuk mencari kesimpulan tersembunyi, menjawab kueri yang sangat kompleks, dan memverifikasi aturan-aturan rekayasa yang dinamis.

10

Python (Algoritma & Aplikasi)

Dalam kerangka Integrasi Teknologi untuk platform *Smart Engineering*, **Python beroperasi sebagai pilar utama untuk eksekusi algoritma prosedural, komputasi numerik, dan pengembangan aplikasi.** Sementara Prolog secara khusus digunakan sebagai mesin penalaran simbolik, Python melengkapinya dengan keserbagunaan dan ekosistem pustakanya (*libraries*) yang sangat luas yang mencakup sains data, komputasi matematis, dan pembuatan antarmuka pengguna (UI).

Di dalam arsitektur sistem terintegrasi (berada pada lapisan Domain Sistem / I(S)), Python memiliki peran dan alur kerja spesifik sebagai berikut:

1. Python sebagai Orkestrator dan Pengendali Alur Kerja Dalam integrasi ini, Python bertindak sebagai “pengendali utama” yang mengatur interaksi pengguna, melakukan pemrosesan data awal (*preprocessing*), dan mengelola alur kerja aplikasi secara keseluruhan. Python difokuskan untuk menangani tugas-tugas prosedural dan algoritmik murni yang merupakan keunggulannya, sementara tugas deduksi logika, evaluasi kendala (*constraints*), dan kueri berbasis pengetahuan “didelegasikan” secara eksklusif ke mesin Prolog di latar belakang.

2. Membangun Simulasi yang Dinamis (Dynamic Interaction) Kekuatan komputasi Python memungkinkan arsitektur ini untuk menjalankan simulasi rekayasa yang sangat dinamis. Alur kerja integrasi ini berjalan secara berkesinambungan:

- **Inisialisasi:** Skrip Python bertugas menginisialisasi sesi Prolog dan memuat berkas basis pengetahuan ontologis (misalnya . `Pl`) ke dalam memori.

- **Manipulasi Fakta Dinamis:** Python dapat secara dinamis memodifikasi basis pengetahuan Prolog saat aplikasi atau simulasi sedang berjalan (*runtime*). Menggunakan fungsi antarmuka, Python bisa menyuntikkan fakta baru (melalui `assertz`) yang merepresentasikan status sistem atau input lingkungan terkini, serta menariknya kembali (`retract`) saat status tersebut sudah tidak relevan.
- **Eksekusi dan Visualisasi:** Python menyusun kueri logika berupa *string* untuk dikirimkan ke Prolog. Setelah Prolog melakukan penalaran, Python bertugas mengekstraksi solusi tersebut, memprosesnya dengan algoritma numerik lanjutan, dan menampilkannya melalui visualisasi atau antarmuka aplikasi.

3. Konversi Data secara Mulus di Tingkat Teknologi (T) Agar kemampuan aplikasi Python dan penalaran Prolog bisa menyatu, integrasi ini sangat bergantung pada keberadaan teknologi jembatan (*bridge libraries* / I(T)) seperti **PySWIP** atau **Janus**. Pustaka inilah yang memungkinkan program Python berinteraksi dengan mulus melalui **manajemen konversi tipe data (data marshalling)**. Sebagai contoh, pustaka ini secara otomatis mengubah struktur *string*, *integer*, atau *list* di dalam algoritma Python menjadi bentuk *atom* atau *term* yang valid bagi Prolog, dan sebaliknya mengubah variabel hasil deduksi Prolog menjadi struktur *dictionary* di Python.

Secara keseluruhan, pendelegasian arsitektural yang menempatkan Python sebagai mesin aplikasi algoritma dan Prolog sebagai mesin logika menghasilkan landasan yang kokoh bagi *Smart Engineering*. Sinergi ini (*Outcome* / O(S) & O(T)) memungkinkan insinyur untuk merancang simulasi dan perangkat lunak yang bukan hanya unggul dalam komputasi matematis (*number-crunching*), tetapi juga memiliki pemahaman terkomputasi yang adaptif dan “sadar-konteks” terhadap model konseptual dunia nyata.

10.1. Pemrosesan Data

Dalam arsitektur terintegrasi Prolog-Python untuk *Smart Engineering*, **Python beroperasi sebagai pilar utama untuk eksekusi algoritma prosedural, komputasi numerik, dan pengembangan aplikasi**. Sementara Prolog difokuskan pada penalaran simbolis dan deduksi logika, Python mengambil alih seluruh beban kerja yang berkaitan dengan **pemrosesan data (data processing/handling)**.

Peran dan alur kerja pemrosesan data oleh Python di dalam ekosistem ini mencakup beberapa aspek utama:

1. Pra-pemrosesan Data dan Orkestrasi Alur Kerja Python bertindak sebagai pengatur utama (*orchestrator*) di lapisan aplikasi dan sistem. Tugas utamanya mencakup **mengelola interaksi pengguna, menangani input/output data (I/O), serta melakukan pra-pemrosesan data (data preprocessing)** sebelum mendelegasikan tugas-tugas penalaran atau kueri pengetahuan yang kompleks kepada mesin Prolog.

2. Komputasi Numerik dan Analisis Data Berbeda dengan Prolog yang unggul dalam logika deklaratif, Python mewakili pendekatan algoritmik tradisional yang **sangat unggul dalam komputasi numerik dan pemrosesan data bervolume besar**. Dengan memanfaatkan ekosistem pustakanya (*libraries*) yang luas di bidang sains data, Python mengeksekusi perhitungan matematis, pemrosesan prosedural, dan analisis data numerik yang tidak ideal untuk diselesaikan oleh mesin inferensi logika.

3. Pertukaran dan Konversi Tipe Data (Data Marshalling) Salah satu tantangan terbesar dalam pemrosesan data lintas bahasa adalah mengelola aliran data antara domain komputasi (Python) dan domain simbolis (Prolog). Integrasi ini bergantung pada modul jembatan (*bridge libraries* seperti PySWIP atau Janus) yang memfasilitasi komunikasi data secara transparan. Pustaka ini **bertanggung jawab secara otomatis mengonversi struktur data Python (seperti string, integer, float, list, dan dictionary) menjadi tipe data Prolog (atom, number, term, list)**, dan begitu pula sebaliknya saat menerima hasil evaluasi dari Prolog. Konversi yang mulus ini membebaskan insinyur dari kerumitan melakukan serialisasi data secara manual.

4. Pemrosesan Hasil Inferensi dan Visualisasi Setelah Prolog selesai mengevaluasi sebuah fakta atau mendeduksi aturan logis, hasil atau solusi tersebut dikembalikan kepada Python. **Python bertugas mengambil hasil kueri ini, lalu memprosesnya lebih lanjut menggunakan algoritma numerik, memanfaatkannya untuk pengambilan keputusan aplikasi, atau menampilkannya kepada pengguna melalui visualisasi data dan antarmuka pengguna (UI).**

Secara keseluruhan, pemosisian Python khusus untuk pemrosesan data dan komputasi matematis menciptakan arsitektur perangkat lunak rekayasa yang sangat modular. Sinergi ini memastikan bahwa sistem dapat **menghitung secara presisi sekaligus menalar secara konseptual**, menghasilkan perangkat lunak yang jauh lebih cerdas, adaptif, dan mampu mengorkestrasi alur kerja rekayasa dunia nyata.

10.2. Antarmuka Pengguna

Dalam arsitektur terintegrasi Prolog-Python untuk platform *Smart Engineering*, Python secara eksklusif mengemban tanggung jawab pada lapisan aplikasi, yang salah satu fokus utamanya adalah pengembangan Antarmuka Pengguna (User Interface / UI) serta interaksi pengguna.

Sementara Prolog bekerja di latar belakang sebagai mesin penalaran simbolis dan basis pengetahuan, Python bertindak sebagai “wajah” dari sistem yang berhadapan langsung dengan manusia. Posisi dan peran antarmuka pengguna ini dijabarkan sebagai berikut:

- **Pengelola Interaksi dan Alur Kerja Utama:** Aplikasi yang dibangun dengan Python bertugas untuk **mengelola seluruh interaksi pengguna, mengumpulkan input atau parameter awal, dan mengatur alur kendali (control flow)**. Ketika pengguna memberikan perintah melalui antarmuka Python, aplikasi akan meneruskan data tersebut kepada mesin Prolog untuk dievaluasi logika atau kendalanya (*constraints*), untuk kemudian ditarik kembali hasil inferensinya.
- **Visualisasi dan Presentasi Hasil:** Setelah mesin Prolog menyelesaikan evaluasi logisnya, kode Python akan mengambil solusi tersebut untuk diekstrak dan diproses lebih lanjut. Tugas akhir dari antarmuka pengguna Python adalah **menyajikan kembali hasil deduksi dan komputasi tersebut kepada pengguna dalam format antarmuka visual yang mudah dipahami**.
- **Pemanfaatan Ekosistem Pustaka (Libraries) yang Kaya:** Fleksibilitas Python menjadikannya pilihan ideal untuk pengembangan aplikasi karena ia didukung oleh ekosistem pustaka yang sangat luas yang mencakup pengembangan web, visualisasi data, dan Antarmuka Pengguna Grafis (GUI).
- **Pengembangan Simulasi yang Interaktif:** Dalam studi kasus pembuatan alat simulasi rekayasa yang lebih canggih, antarmuka pengguna grafis (GUI) dapat dibangun **menggunakan pustaka Python spesifik seperti Dash, Plotly, atau Streamlit**. Keberadaan UI yang interaktif ini sangat memudahkan insinyur atau pengguna dalam memasukkan parameter skenario, memilih berbagai alternatif solusi, serta memvisualisasikan hasil simulasi secara dinamis.

Secara keseluruhan, pendelegasian pengembangan antarmuka pengguna kepada Python memastikan bahwa kerumitan ontologi dan logika formal di dalam

Prolog dapat disembunyikan, lalu diubah menjadi perangkat lunak rekayasa yang interaktif, visual, dan ramah pengguna di dunia nyata.

10.3. Orkestrasi Simulasi

Dalam ekosistem platform *Smart Engineering* yang terintegrasi, **Python mengambil peran sentral sebagai orkestrator yang mengendalikan dan menjalankan keseluruhan alur simulasi**. Sementara Prolog beroperasi murni sebagai fondasi basis pengetahuan dan mesin penalaran logis, Python menyediakan aliran kendali imperatif (*imperative control flow*) serta kemampuan komputasi numerik yang mutlak dibutuhkan untuk “menghidupkan” ontologi tersebut menjadi sebuah simulasi yang berjalan.

Berdasarkan sumber-sumber yang ada, proses orkestrasi simulasi oleh Python mencakup beberapa tahapan dan fungsi krusial:

- **Inisialisasi dan Persiapan Skenario:** Skrip Python bertindak sebagai pengelola utama yang memulai alur kerja dengan menginisialisasi sesi Prolog dan memuat berkas ontologi (misalnya file `.pl`) ke dalam memori mesin. Pada tahap ini, Python juga bertugas mendefinisikan parameter-parameter skenario. Sebagai contoh, dalam studi kasus simulasi transportasi koridor Bandung-Jakarta 2030, Python digunakan untuk merumuskan beban skenario seperti rute sepanjang 150 km, tahun proyeksi 2030, dan permintaan jumlah penumpang sebesar 100 orang.
- **Penggerak Simulasi Dinamis (Dynamic State Modification):** Sebuah simulasi yang sesungguhnya harus mampu merepresentasikan perubahan status seiring berjalannya waktu. Python mengorkestrasi dinamika ini dengan **secara aktif menyuntikkan (meng-assert) fakta-fakta baru ke dalam basis pengetahuan Prolog saat simulasi sedang berjalan (runtime), serta menariknya kembali (retract) ketika status tersebut sudah tidak relevan**. Kemampuan manipulasi interaktif inilah yang memungkinkan status dari model Prolog untuk terus berevolusi merespons peristiwa dan data skenario tanpa harus memodifikasi kode dasar Prolog untuk setiap skenario yang berbeda.
- **Berperan sebagai Mesin Kalkulasi (Calculation Engine):** Sebagai orkestrator, Python mendelegasikan pengambilan data logis ke Prolog, lalu menarik hasil kuerinya untuk diproses menggunakan komputasi algoritmik. Dalam simulasi transportasi, kelas-kelas pada Python mengambil data properti kendaraan dan energi secara langsung dari Prolog (melalui pustaka *bridge* PySWIP). Setelah data logis didapatkan, algoritma Python menghitung secara matematis jumlah unit armada yang dibutuhkan, total

10. Python (Algoritma & Aplikasi)

estimasi biaya per penumpang, serta proyeksi emisi CO₂ berdasarkan konsumsi energi dan jarak tempuh.

- **Eksplorasi Skenario Lanjutan dan Optimasi:** Keunggulan algoritma Python memungkinkan orkestrasi skenario yang jauh lebih kompleks. Sumber menyarankan bahwa melalui Python, simulasi dapat diperluas untuk memasukkan **pemodelan ketidakpastian (seperti simulasi Monte Carlo), analisis sensitivitas, hingga algoritma optimasi (seperti algoritma genetik)** guna menemukan konfigurasi sistem transportasi yang paling ideal berdasarkan kriteria keberhasilan (energi PSKVE) yang diharapkan.

Secara keseluruhan, pemisahan arsitektur yang jelas—di mana representasi pengetahuan dan aturan logika diisolasi di Prolog, sementara logika simulasi, komputasi, dan kontrol alur diorkestrasi sepenuhnya oleh Python—menghasilkan **peningkatan modularitas dan kemudahan pemeliharaan (maintainability) dari perangkat lunak rekayasa**. Pendekatan hibrida ini sangat ideal untuk memodelkan sistem *Smart Engineering* yang kompleks karena menggabungkan kecerdasan komputasi matematis dengan kesadaran logis (*knowledge-aware*).

11

Bridge Libraries

Dalam kerangka Integrasi Teknologi untuk platform *Smart Engineering*, **Pustaka Penghubung (Bridge Libraries)** merupakan instrumen teknologi kunci (I(T)) yang beroperasi pada Domain Teknologi untuk menyatukan dua paradigma pemrograman yang bertolak belakang: penalaran simbolik di Prolog dan komputasi algoritmik di Python. Pustaka ini secara khusus memfasilitasi komunikasi dan pertukaran data antar-bahasa, sekaligus mengabstraksi kerumitan pemanggilan fungsi lintas-proses tingkat rendah.

Peran, fungsionalitas, dan posisi *bridge libraries* di dalam integrasi ini dijabarkan sebagai berikut:

- 1. Manajemen Konversi Tipe Data yang Transparan (Data Marshalling)** Komunikasi yang efektif antara Python dan Prolog sangat bergantung pada kemampuan untuk mengonversi tipe data secara mulus. *Bridge libraries* memikul tanggung jawab krusial ini dengan memetakan struktur data secara otomatis tanpa mengharuskan insinyur melakukan serialisasi data secara manual. Sebagai contoh, pustaka ini secara otomatis mengubah *string*, *integer*, *float*, dan *list* dari Python menjadi bentuk *atom*, *number*, dan *term* yang dipahami Prolog, lalu mengonversi kembali variabel hasil deduksi Prolog menjadi struktur seperti *dictionary* di dalam Python.
- 2. Menjembatani Orkestrasi Alur Kerja (Calling Conventions)** Pustaka ini menyediakan antarmuka teknis yang mengizinkan Python untuk sepenuhnya mengendalikan dan berinteraksi dengan mesin Prolog. Melalui pustaka ini, skrip Python dapat memulai sesi Prolog, memuat berkas ontologi (`.pl`),

11. Bridge Libraries

secara dinamis menyuntikkan atau menarik fakta saat simulasi berjalan (seperti `assertz` atau `retract`), serta mengirimkan rangkaian kueri logika untuk dievaluasi oleh mesin inferensi Prolog.

Wujud Nyata Bridge Libraries Sumber-sumber yang ada menyoroti beberapa pustaka penghubung yang paling menonjol:

- **PySWIP:** Pustaka yang sangat populer untuk menghubungkan Python dengan SWI-Prolog dengan memanfaatkan pustaka fungsi eksternal *ctypes* milik Python. PySWIP memberikan antarmuka bergaya *Pythonic* yang memudahkan programmer melakukan kueri dan mengelola hasil deduksi berupa *list* dari *dictionary*.
- **Janus:** Pustaka yang dirancang khusus untuk memberikan antarmuka komunikasi dua arah (*bi-directional*) yang berkinerja sangat tinggi untuk SWI-Prolog, XSB, dan Ciao Prolog. Dibangun di atas API bahasa C tingkat rendah, Janus menawarkan latensi komunikasi hanya sekitar satu mikrodetik dan mendukung iterasi non-deterministik antar-bahasa.
- **Langchain-Prolog & prologmqi:** Pustaka lanjutan di mana Langchain-Prolog mengintegrasikan penalaran logis SWI-Prolog ke dalam aplikasi Kecerdasan Buatan Generatif (LLM), sementara *prologmqi* memfasilitasi komunikasi berbasis antrean pesan (*message queue*) dengan pertukaran data format JSON.

Dampak Sistemik (Outcome / $O(T)$) Kehadiran pustaka jembatan ini secara signifikan menurunkan hambatan teknis bagi insinyur untuk memadukan kecerdasan logis ke dalam aplikasi perangkat lunak mereka. Pustaka ini memastikan aliran komunikasi yang sangat efisien dan meminimalkan beban komputasi (*overhead*) saat menerjemahkan struktur data antar-bahasa. Pada akhirnya, inovasi yang terjadi di lapisan Domain Teknologi inilah yang menjadi fondasi penentu sehingga platform sistem integrasi multi-domain—yang menyatukan kehebatan komputasi numerik Python dengan pemahaman basis pengetahuan Prolog—dapat benar-benar diwujudkan.

11.1. PySWIP

Dalam kerangka Integrasi Teknologi, Pustaka Penghubung (*Bridge Libraries*) berperan sebagai instrumen teknologi kunci di Domain Teknologi (I(T)) yang bertugas menyatukan komputasi algoritmik Python dengan penalaran simbolik Prolog. Di antara berbagai opsi pustaka yang ada, **PySWIP** merupakan salah satu pustaka penghubung yang paling populer dan secara luas diandalkan.

Berikut adalah penjabaran mengenai posisi dan kemampuan PySWIP di dalam ekosistem integrasi teknologi ini:

1. Menjembatani Python secara Khusus dengan SWI-Prolog PySWIP dirancang secara spesifik untuk menghubungkan aplikasi Python dengan **SWI-Prolog**, yakni salah satu implementasi Prolog sumber terbuka (*open-source*) yang paling tangguh dan banyak digunakan. Secara teknis, PySWIP beroperasi dengan cara menjadikan SWI-Prolog sebagai pustaka bersama (*shared library*). Python kemudian mengakses pustaka tersebut dengan memanfaatkan modul eksternalnya, yaitu `ctypes` (*foreign function library*).

2. Menyediakan Antarmuka Bergaya Pythonic Fungsi utama dari PySWIP adalah menyembunyikan kerumitan komunikasi antar-proses (*inter-process communication*) dan memberikan antarmuka bergaya *Pythonic* kepada pengembang. Dengan menggunakan PySWIP, skrip Python dapat secara mandiri mengontrol mesin Prolog melalui perintah-perintah langsung, seperti:

- **Inisialisasi:** Membuat instansiasi mesin Prolog dari dalam Python (misalnya `prolog = Prolog()`).
- **Pemuatan Berkas:** Memuat berkas ontologi atau basis pengetahuan berekstensi `.pl` ke dalam memori mesin logika menggunakan perintah seperti `prolog.consult()`.

3. Pertukaran Data yang Transparan (Data Marshalling) Sebagai *bridge library*, PySWIP memegang peranan vital dalam mengatur pertukaran tipe data. Saat Python mengirimkan kueri dan Prolog berhasil menemukan solusinya, PySWIP secara otomatis mengonversi variabel-variabel temuan Prolog tersebut menjadi **sebuah struktur list (daftar) yang berisi dictionary (kamus) di dalam lingkungan Python**. Pada struktur *dictionary* ini, kuncinya (*keys*) merepresentasikan nama variabel dalam kueri Prolog, sementara nilainya (*values*) berisi temuan logis yang saling terikat. Konversi data yang transparan ini sangat krusial karena ia memungkinkan algoritma numerik Python langsung memproses hasil deduksi logika tersebut.

4. Penggerak Simulasi Rekayasa yang Dinamis Dalam konteks pengembangan aplikasi atau simulasi rekayasa yang dinamis (*Smart Engineering*), PySWIP adalah alat yang memungkinkan Python bertindak sebagai orkestrator yang mengubah status “dunia” di dalam Prolog secara *real-time* :

- **Penyuntikan dan Penarikan Fakta:** Melalui fungsi seperti `prolog.assertz()`, Python dapat menyuntikkan data baru ke dalam Prolog untuk merepresentasikan kejadian atau status baru. Setelah evaluasi selesai, Python

juga dapat mencabut fakta tersebut menggunakan `prolog. Retract()` agar basis pengetahuan kembali bersih.

- **Simulasi Multi-Domain:** Dalam studi kasus simulasi moda transportasi koridor Bandung-Jakarta 2030, kelas-kelas kalkulasi di dalam Python memanfaatkan antarmuka PySWIP untuk mengkueri properti kendaraan yang tersimpan di ontologi Prolog (misalnya atribut `vehicleproperty(...)`). Data ontologis yang telah diikat ini kemudian digunakan oleh Python untuk menghitung kebutuhan biaya transportasi dan emisi gas buang secara matematis.

Singkatnya, **PySWIP adalah instrumen teknologi inti yang mewujudkan arsitektur integrasi multi-paradigma**. Ia menjadi jembatan eksekusi yang memungkinkan insinyur membangun aplikasi Python yang sangat fleksibel, interaktif, dan komputasional, yang ditenagai oleh “otak” pemahaman deduktif tingkat tinggi dari mesin SWI-Prolog di latar belakang.

11.2. Janus (SWI-Prolog)

Dalam kerangka Integrasi Teknologi, Pustaka Penghubung (*Bridge Libraries*) berfungsi sebagai instrumen teknologi vital yang mengabstraksi kerumitan pemanggilan fungsi lintas-bahasa untuk menyatukan komputasi algoritmik Python dengan penalaran simbolik Prolog. Di dalam ekosistem pustaka penghubung ini, **Janus menonjol sebagai antarmuka komunikasi tingkat lanjut yang dirancang secara khusus untuk memfasilitasi interaksi dua arah (bi-directional) dengan kinerja yang sangat tinggi**.

Berikut adalah karakteristik dan peran spesifik Janus di dalam konteks pustaka penghubung:

- **Komunikasi Dua Arah Berkecepatan Tinggi:** Janus dibangun langsung di atas antarmuka pemrograman aplikasi (API) tingkat rendah dari bahasa C untuk kedua lingkungan pemrograman. Pendekatan arsitektur ini menghasilkan jalur komunikasi yang sangat efisien, dengan **latensi yang dilaporkan mencapai sekitar satu mikrodetik**.
- **Dukungan Multi-Implementasi:** Tidak hanya dirancang untuk SWI-Prolog, antarmuka Janus juga memperluas kompatibilitasnya untuk dapat diintegrasikan dengan implementasi Prolog yang tangguh lainnya, seperti XSB Prolog dan Ciao Prolog.
- **Fungsionalitas Pemanggilan Timbal Balik:** Janus memberikan fleksibilitas penuh di mana kedua bahasa dapat saling mengendalikan satu sama lain:

- **Dari Python ke Prolog:** Python dapat menggunakan fungsi seperti `janus.Query_once()` dan `janus.Apply_once()` untuk mengeksekusi kueri deterministik (solusi tunggal). Untuk predikat non-deterministik yang dapat menghasilkan banyak solusi, Janus menyediakan antarmuka iterator seperti `janus.Query()` dan `janus.Apply()`.
- **Dari Prolog ke Python:** Sebaliknya, Prolog dapat menggunakan predikat seperti `py_call/2` untuk memanggil dan mengeksekusi fungsi Python secara langsung, serta menggunakan `py_iter/2` untuk menelusuri data dari iterator Python.
- **Manajemen Konversi Data (Data Marshalling) yang Sangat Rinci:** Keberhasilan integrasi sistem sangat bergantung pada kelulusan konversi tipe data. Janus menyembunyikan kerumitan serialisasi dengan menyediakan peta terjemahan tipe data transparan yang komprehensif :
 - Nilai numerik (*integer* dan *float*) di Prolog secara langsung menjadi tipe data `int` (termasuk *big integers*) dan `float` di Python.- Atom Prolog dapat dikonversi secara fleksibel menjadi *string* standar atau instansiasi *class enum*. `Enum` di Python.- Struktur *list* dan *dict* (khusus di SWI-Prolog) diubah secara otomatis menjadi struktur *list* dan *dictionary* milik Python.- Janus menangani atom khusus di Prolog seperti `@(true)`, `@(false)`, dan `@(none)` dan memetakannya secara persis menjadi objek boolean `True`, `False`, dan `None` di Python.
- **Fondasi bagi Aplikasi Kecerdasan Buatan (LLM):** Keandalan Janus menjadikannya sebagai fondasi teknologi bagi pengembangan pustaka tingkat tinggi seperti **Langchain-Prolog**. Pustaka lanjutan ini memanfaatkan Janus untuk mengintegrasikan mesin penalaran SWI-Prolog ke dalam LangChain, sebuah kerangka kerja untuk membangun aplikasi berbasis Kecerdasan Buatan Generatif (*Large Language Models*). Hal ini memungkinkan mesin AI untuk memadukan penalaran logis terstruktur dengan kapabilitas bahasa generatif.

Secara keseluruhan, kehadiran Janus meningkatkan standar *bridge libraries* di dalam praktik *Smart Engineering*. Dengan mengotomatisasi konversi tipe data yang rumit dan menyediakan latensi komunikasi yang sangat rendah, **Janus memastikan bahwa penalaran logika Prolog dan algoritma prosedural Python benar-benar terintegrasi sebagai sebuah mesin komputasi tunggal yang kohesif.**

11.3. PrologMQI

Dalam kerangka Integrasi Teknologi, **prologmqi** beroperasi di tingkat Domain Teknologi (I(T)) sebagai salah satu alternatif Pustaka Penghubung (*Bridge Libraries*) untuk mengintegrasikan mesin komputasi Python dengan mesin penalaran SWI-Prolog.

Meskipun memiliki tujuan yang sama dengan pustaka penghubung lain seperti PySWIP atau Janus, **prologmqi** menawarkan pendekatan arsitektur yang sangat berbeda di dalam ekosistem integrasi ini:

1. Pendekatan Berbasis Antrean Pesan (Message Queue) Alih-alih menghubungkan Python dan Prolog pada tingkat memori yang sama (seperti menggunakan API bahasa C pada Janus), **prologmqi** memanfaatkan antarmuka kueri mesin (*machine query interface*) dengan pendekatan **antrean pesan (message queue)**. Dalam mekanisme ini, aplikasi Python mengeksekusi kueri SWI-Prolog dengan cara mengirimkan instruksi tersebut dalam bentuk *string* murni.

2. Pertukaran Data Menggunakan JSON Untuk menangani tantangan konversi tipe data lintas bahasa (*data marshalling*), **prologmqi** tidak melakukan pemetaan objek secara langsung di memori, melainkan **mengembalikan respons atau hasil deduksi dari Prolog dalam format JSON**. Penggunaan JSON ini mengabstraksi kerumitan penerjemahan data karena JSON merupakan standar pertukaran data yang sangat umum dan mudah diproses ulang oleh algoritma Python.

3. Keunggulan untuk Sistem Terdistribusi Di dalam konteks yang lebih luas mengenai bagaimana para insinyur merancang arsitektur sistem (*Smart Engineering*), pemilihan pustaka penghubung bergantung pada kebutuhan desain. Sementara pustaka seperti Janus dioptimalkan untuk komunikasi internal dua arah dengan latensi sangat rendah, karakteristik **prologmqi** (berbasis pesan dan JSON) **menjadikannya sangat cocok dan ideal untuk membangun arsitektur sistem yang terdistribusi (distributed architectures) atau sistem yang digabungkan secara longgar (loosely coupled architectures)**.

Dengan kata lain, **prologmqi** memungkinkan pengembang untuk menjalankan mesin logika Prolog dan mesin aplikasi Python pada server, wadah (*container*), atau proses yang sepenuhnya terpisah, di mana keduanya saling berkomunikasi secara asinkron sebagai layanan yang independen.

Bagian 4: Studi Kasus

12

Studi Kasus: Transportasi 2030

Studi Kasus Transportasi koridor Bandung-Jakarta 2030 adalah sebuah simulasi praktis yang dirancang secara khusus untuk mendemonstrasikan kegunaan kerangka kerja ontologis *Smart Engineering* dalam menyelesaikan masalah rekayasa multi-domain yang kompleks.

Dalam konteks yang lebih luas dari *Smart Engineering*, studi kasus ini menyoroti beberapa penerapan prinsip fundamental:

1. Implementasi Nyata Integrasi Prolog-Python Simulasi ini menjadi bukti nyata dari penerapan arsitektur sistem (Domain Sistem) yang memadukan komputasi algoritmik dan penalaran simbolis.

- **Prolog sebagai Basis Pengetahuan:** Model ontologi terkait konsep komponen tahun 2030—seperti spesifikasi *Electric Sedan*, bus *biodiesel*, kereta cepat, kapasitas penumpang, biaya jaringan listrik, hingga intensitas CO₂—didefinisikan secara deklaratif di dalam basis pengetahuan Prolog.
- **Python sebagai Orkestrator dan Mesin Kalkulasi:** Python bertugas menjalankan logika skenario (misalnya target membawa 100 penumpang). Melalui pustaka penghubung seperti `pyswip`, *class* di Python menarik data properti kendaraan dari Prolog, lalu menggunakan algoritma matematisnya untuk menghitung jumlah armada yang dibutuhkan, mengalkulasi biaya total operasional, dan menghitung emisi secara keseluruhan. Pemisahan yang jelas antara representasi pengetahuan (Prolog) dan logika simulasi (Python) ini terbukti sangat berharga untuk meningkatkan modularitas dan kemudahan pemeliharaan sistem perangkat lunak.

2. Pergeseran Metrik Evaluasi menuju Dimensi Energi PSKVE Alih-alih sekadar membandingkan spesifikasi teknis mesin fisik, *Smart Engineering* memandang solusi rekayasa dari lensa multi-dimensi PSKVE (Produk, Layanan, Pengetahuan, Nilai, Lingkungan). Kinerja ketiga alternatif solusi transportasi dalam studi kasus ini dievaluasi melalui matriks energi tersebut:

- **Energi Layanan (Service Energy):** Direpresentasikan secara objektif melalui efisiensi waktu atau estimasi waktu tempuh perjalanan, di mana Kereta Cepat menunjukkan performa Energi Layanan terbaik.
- **Energi Nilai (Value Energy):** Diukur melalui total biaya yang harus dikeluarkan per penumpang. Parameter algoritmik di Python mengakumulasi biaya ini berdasarkan harga energi, tarif tol, nilai tiket, hingga pro-rata biaya pemeliharaan mesin.
- **Energi Ruang Lingkungan (Environmental Space Energy):** Mewakili jejak ekologis dari teknologi tersebut, yang dalam simulasi ini dikalkulasi sebagai jumlah gram emisi CO₂ ekuivalen per penumpang.

3. Mengilustrasikan Prinsip Konversi Transaksional Studi kasus ini juga memperlihatkan cara kerja Konversi Transaksional (*Transactional Conversion*) di dunia nyata. Sistem transportasi ini memodelkan bagaimana pemangku kepentingan (penumpang) melakukan transaksi energi dimensi silang: **mereka menukarkan Energi Nilai yang mereka miliki (berupa uang/biaya tiket) untuk mendapatkan Energi Layanan (akses transportasi yang efisien) serta memanfaatkan konversi Energi Produk (pengoperasian mekanis kendaraan).**

4. Menggambarkan Interaksi Multi-Domain yang Rumit Dengan mengevaluasi kendaraan yang berbeda (armada mobil listrik pribadi, bus diesel, dan kereta cepat), simulasi ini mendemonstrasikan kompleksitas interaksi dari berbagai “wares” (perangkat keras kendaraan, perangkat lunak sistem otonom, hingga infrastruktur jaringan energi). Sebagai contoh, simulasi ini membuktikan bahwa **Energi Pengetahuan (Knowledge Energy)** yang disematkan dalam teknologi baterai *Electric Vehicle* (EV) atau manajemen jaringan listrik cerdas akan secara langsung mendikte seberapa efisien konsumsi **Energi Produk** kendaraan tersebut, dan pada akhirnya menentukan besaran **Energi Ruang Lingkungan** (emisi) yang dihasilkannya.

Secara keseluruhan, simulasi rute Bandung-Jakarta 2030 ini bukan sekadar studi kasus transportasi biasa, melainkan cetak biru (*blueprint*) tentang bagaimana kerangka kerja *Smart Engineering* memberdayakan para insinyur untuk merancang, membandingkan, dan mengoptimalkan sistem masa depan secara holistik dengan memadukan basis pengetahuan kecerdasan buatan, perhitungan matematis algoritmik, dan valuasi multi-dimensi PSKVE.

12.1. Skenario Bandung-Jakarta

Dalam kerangka Studi Kasus Transportasi 2030, **Skenario Bandung-Jakarta** merupakan sebuah model simulasi praktis yang dirancang untuk mendemonstrasikan bagaimana kerangka kerja ontologis *Smart Engineering* dapat diimplementasikan untuk mengevaluasi alternatif solusi sistem rekayasa multi-domain.

Berikut adalah rincian dari Skenario Bandung-Jakarta dan signifikansinya di dalam konteks studi kasus yang lebih luas:

1. Batasan dan Parameter Skenario Untuk menjalankan simulasi yang terukur, skenario ini menetapkan tiga parameter operasional utama:

- **Rute Perjalanan:** Koridor dari Bandung menuju Jakarta dengan estimasi jarak tempuh 150 km.
- **Tahun Proyeksi:** Kondisi teknologi, harga energi, dan infrastruktur disimulasikan untuk tahun 2030.
- **Beban Permintaan (Passenger Demand):** Skenario ini menargetkan pengangkutan sebuah kelompok yang terdiri dari **100 orang penumpang** secara bersamaan.

2. Alternatif Transportasi yang Dievaluasi Skenario ini membandingkan tiga jenis teknologi kendaraan massal dan pribadi (yang datanya tersimpan di dalam basis pengetahuan Prolog) untuk memenuhi permintaan 100 penumpang tersebut:

- **Armada Sedan Listrik (Electric Sedan):** Memiliki kapasitas 4 penumpang per unit, sehingga skenario ini membutuhkan kalkulasi pengoperasian 25 unit mobil listrik.
- **Armada Bus Diesel Euro 6 (Biodiesel):** Memiliki kapasitas 40 penumpang per unit, sehingga skenario ini membutuhkan pengoperasian 3 unit bus.
- **Kereta Cepat Listrik (High-Speed Train):** Memiliki kapasitas masif sebesar 500 penumpang per rangkaian, sehingga cukup menggunakan satu layanan untuk mengangkut seluruh kelompok.

3. Evaluasi Berdasarkan Metrik Energi PSKVE Alih-alih membandingkan spesifikasi mesin secara mentah, skenario Bandung-Jakarta mengevaluasi kinerja ketiga solusi tersebut menggunakan lensa multi-dimensi Energi PSKVE:

- **Energi Layanan (Service Energy):** Direpresentasikan melalui estimasi waktu tempuh. Kereta Cepat menawarkan efisiensi energi layanan terbaik (sekitar 1 hingga 1,5 jam) dibandingkan opsi jalan raya.
- **Energi Nilai (Value Energy):** Direpresentasikan melalui total biaya operasional per penumpang. Dalam simulasi algoritma Python, biaya ini diakumulasi dari harga energi/bahan bakar, tarif tol, hingga prorata biaya pemeliharaan. Hasil ilustratif menunjukkan Bus Diesel memiliki potensi biaya per penumpang termurah, diikuti oleh armada Sedan EV, sedangkan Kereta Cepat memiliki biaya langsung tertinggi melalui harga tiket.
- **Energi Ruang Lingkungan (Environmental Space Energy):** Direpresentasikan dari total emisi ekuivalen CO₂ per penumpang. Kereta Cepat menghasilkan jejak karbon terendah, sementara emisi Sedan EV sangat bergantung pada intensitas karbon jaringan listrik PLN di tahun 2030, dan Bus Diesel mencatatkan hasil polusi tertinggi.

4. Signifikansi dalam Konteks Smart Engineering Skenario Bandung-Jakarta ini berfungsi sebagai pembuktian konsep (*proof of concept*) dari dua prinsip utama *Smart Engineering*:

- **Pembuktian Integrasi Python-Prolog:** Simulasi ini memvalidasi kegunaan pemisahan arsitektur sistem. Data sifat kendaraan (seperti kapasitas dan konsumsi energi) diisolasi secara rapi di dalam pangkalan pengetahuan Prolog, sementara skrip Python secara dinamis menarik data tersebut untuk mengeksekusi kalkulasi matematis (seperti menghitung jumlah armada yang dibutuhkan untuk 100 penumpang dan total biaya gabungan).
- **Pembuktian Konversi Transaksional:** Skenario ini mengilustrasikan secara konkret bagaimana pengguna (penumpang) melakukan pertukaran energi lintas-dimensi di dunia nyata: mereka menukarkan Energi Nilai (uang/biaya) untuk mendapatkan Energi Layanan (waktu perjalanan yang efisien) dengan mempekerjakan Energi Produk mekanis kendaraan.

12.2. Evaluasi Fleet EV vs Bus vs Kereta Cepat

Dalam kerangka Studi Kasus Transportasi 2030, evaluasi antara **Armada Sedan EV (Electric Vehicle)**, **Armada Bus Diesel**, dan **Kereta Cepat** merupakan model simulasi yang krusial untuk mendemonstrasikan bagaimana kerangka kerja ontologis *Smart Engineering* membandingkan berbagai alternatif solusi secara holistik. Skenario ini membandingkan ketiga moda tersebut untuk memenuhi permintaan pengangkutan 100 orang penumpang di koridor rute Bandung-Jakarta (150 km).

Berikut adalah perbandingan dan evaluasi komprehensif dari ketiga solusi transportasi tersebut menggunakan lensa dimensi energi PSKVE (Produk, Layanan, Pengetahuan, Nilai, Lingkungan):

1. Konfigurasi Kebutuhan Armada (Domain Sistem) Untuk mengangkut kelompok berisi 100 penumpang, algoritma komputasi Python menarik data properti fisik kendaraan dari ontologi Prolog dan menghitung kebutuhan kapasitas masing-masing solusi:

- **Armada Sedan EV:** Karena setiap mobil berkapasitas 4 orang, skenario ini harus mensimulasikan operasi **25 unit mobil listrik** secara bersamaan.
- **Armada Bus Diesel:** Memiliki kapasitas 40 orang per unit, sehingga skenario ini cukup mengevaluasi **3 unit bus** (yang diasumsikan menggunakan bahan bakar biodiesel B30).
- **Kereta Cepat Elektrik:** Memiliki kapasitas masif sebesar 500 penumpang per rangkaian, sehingga pengoperasian **1 layanan rangkaian** sudah lebih dari cukup untuk memenuhi permintaan target.

2. Evaluasi Energi Layanan (Service Energy) Metrik ini mengukur kinerja layanan berdasarkan efisiensi waktu tempuh perjalanan. Dalam dimensi ini, **Kereta Cepat menawarkan performa yang paling superior**, dengan estimasi waktu tempuh hanya 1,0 hingga 1,5 jam. Sebaliknya, alternatif yang menggunakan infrastruktur jalan raya jauh lebih lambat; Armada Sedan EV membutuhkan waktu sekitar 2,5 hingga 3,5 jam, dan Armada Bus Diesel menjadi opsi paling lambat dengan estimasi 3,0 hingga 4,0 jam perjalanan.

3. Evaluasi Energi Nilai (Value Energy) Metrik ini mengevaluasi pengorbanan finansial melalui akumulasi total biaya per penumpang. Pada simulasi armada berbasis jalan raya, biaya ini dikalkulasi dengan menjumlahkan tarif energi/bahan bakar, tarif tol, serta pembagian prorata biaya pemeliharaan tahunan.

- **Armada Bus Diesel terbukti sebagai solusi paling ekonomis secara biaya operasional langsung**, yang diestimasi sekitar Rp76.381 per penumpang.
- **Armada Sedan EV** menyusul di posisi kedua dengan beban biaya operasional sekitar Rp107.750 per penumpang.
- **Kereta Cepat** merupakan opsi transportasi paling mahal bagi pengguna langsung, yang direpresentasikan oleh harga pembelian tiket sebesar Rp150.000 per penumpang. Meski angka tersebut tampak definitif, sumber mencatat bahwa evaluasi Energi Nilai akan jauh lebih komprehensif jika

simulasi ini turut memperhitungkan biaya siklus hidup (*lifecycle cost*), termasuk pengeluaran modal infrastruktur secara penuh.

4. Evaluasi Energi Ruang Lingkungan (Environmental Space Energy)

Evaluasi jejak karbon diukur dari besaran emisi ekuivalen CO₂ yang dihasilkan untuk memindahkan satu orang penumpang.

- **Kereta Cepat merupakan solusi yang paling ramah lingkungan**, menghasilkan emisi terendah (sekitar 840 gram CO₂eq per penumpang) dengan asumsi suplai dari jaringan listrik yang lebih bersih di tahun 2030.
- **Armada Sedan EV** berada di posisi kedua (sekitar 1.575 gram CO₂eq per penumpang). Evaluasi menekankan bahwa seberapa bersih performa lingkungan mobil listrik sangat bergantung pada intensitas karbon dari jaringan listrik utama (PLN) yang menyuplainya. Hal ini mencontohkan bagaimana Energi Pengetahuan (misalnya, kecerdasan dalam sistem *smart grid*) sangat mendikte dampak pencemaran dari sistem tersebut.
- **Armada Bus Diesel** mencatatkan dampak lingkungan terburuk dengan emisi tertinggi (sekitar 3.234 gram CO₂eq per penumpang), sekalipun sudah dihipotesiskan menggunakan bahan bakar biofuel.

Signifikansi Evaluasi Komparatif Evaluasi antara EV, Bus, dan Kereta Cepat ini secara nyata memvalidasi prinsip **Konversi Transaksional**. Simulasi ini mengilustrasikan sebuah ekosistem *Smart Engineering* di mana penumpang menukarkan Energi Nilai (berupa uang tiket atau biaya tol) untuk mendapatkan Energi Layanan (efisiensi waktu yang tinggi pada kereta cepat, atau akses pada bus murah) dengan mempekerjakan Energi Produk dari operasi mekanis ketiga kendaraan tersebut. Pada akhirnya, simulasi ini membuktikan kemampuan arsitektur hibrida untuk membantu insinyur menimbang ragam kompromi (*trade-offs*) sistem multi-domain secara objektif dan matematis.

12.3. Metrik: Waktu, Biaya, Emisi CO₂

Dalam Studi Kasus Transportasi koridor Bandung-Jakarta 2030, metrik **Waktu, Biaya, dan Emisi CO₂** tidak sekadar dipandang sebagai ukuran teknis tradisional, melainkan digunakan sebagai wujud kuantitatif dari dimensi **Energi PSKVE (Produk, Layanan, Pengetahuan, Nilai, Lingkungan)** di dalam kerangka *Smart Engineering*.

Algoritma komputasi Python secara dinamis mengekstraksi data spesifikasi kendaraan dari ontologi Prolog, lalu menghitung ketiga metrik tersebut untuk mengevaluasi skenario pemenuhan target 100 penumpang melintasi jarak 150 km. Berikut adalah penjabaran ketiga metrik tersebut dalam simulasi ini:

1. Metrik Waktu Tempuh: Representasi Energi Layanan (Service Energy) Waktu tempuh dievaluasi untuk mengukur efisiensi layanan dari sebuah sistem transportasi.

- **Kereta Cepat Elektrik** menawarkan efisiensi Energi Layanan yang paling superior dengan perkiraan waktu tempuh hanya 1,0 hingga 1,5 jam.- Opsi berbasis infrastruktur jalan raya jauh lebih lambat, di mana **Armada Sedan EV** membutuhkan waktu 2,5 hingga 3,5 jam, dan **Armada Bus Diesel** merupakan opsi paling lambat dengan waktu 3,0 hingga 4,0 jam.

2. Metrik Biaya: Representasi Energi Nilai (Value Energy) Metrik biaya mengukur total pengorbanan finansial per penumpang. Untuk armada jalan raya, kalkulator algoritma Python menyimulasikan biaya ini dengan menjumlahkan tarif energi/bahan bakar, tarif tol jalan, serta pembagian secara prorata untuk biaya pemeliharaan kendaraan per tahun. Sementara untuk kereta cepat, harga tiket digunakan sebagai indikator langsung.

- **Armada Bus Diesel** menjadi solusi dengan Energi Nilai paling ekonomis secara langsung, yakni sekitar Rp76.381 per penumpang.
- **Armada Sedan EV** menyusul dengan estimasi beban sekitar Rp107.750 per penumpang.
- **Kereta Cepat** merupakan moda yang menuntut pengeluaran biaya langsung tertinggi sebesar Rp150.000 per penumpang. Sumber mencatat bahwa untuk mendapatkan gambaran Energi Nilai yang lebih utuh, perhitungan metrik biaya di masa depan idealnya turut menyertakan evaluasi biaya siklus hidup (*lifecycle cost*) secara penuh, termasuk biaya modal pembangunan infrastruktur transportasi tersebut.

3. Metrik Emisi CO2: Representasi Energi Ruang Lingkungan (Environmental Space Energy) Metrik ini mengukur jejak karbon dalam wujud gram ekuivalen CO2 per penumpang untuk merepresentasikan dampak ekologis sistem transportasi.

- **Kereta Cepat** kembali unggul dengan emisi terendah (sekitar 840 gram CO2eq per penumpang), diasumsikan didukung oleh jaringan listrik yang lebih bersih.
- **Armada Sedan EV** menghasilkan sekitar 1.575 gram CO2eq per penumpang. Metrik emisi untuk kendaraan listrik ini sangat fluktuatif dan bergantung langsung pada tingkat kebersihan (intensitas karbon) dari jaringan listrik utama (PLN) di tahun 2030.
- **Armada Bus Diesel** mencatatkan dampak lingkungan terburuk dengan memproduksi emisi tertinggi (sekitar 3.234 gram CO2eq per penumpang) meskipun telah menggunakan alternatif biofuel.

Signifikansi dalam Kerangka Smart Engineering Perhitungan ketiga metrik ini digunakan untuk memvalidasi prinsip **Konversi Transaksional (Transactional Conversion)**. Melalui komparasi simulasi ini, sistem *Smart Engineering* mengilustrasikan bagaimana seorang pengguna (penumpang) melakukan transaksi energi dimensi silang di dunia nyata: **mereka menukarkan Energi Nilai (biaya operasional atau tiket) untuk mendapatkan Energi Layanan (efisiensi waktu tempuh) dengan mempekerjakan Energi Produk mekanis kendaraan, yang keseluruhannya meninggalkan jejak Energi Ruang Lingkungan (emisi CO₂).**

Pada akhirnya, pendelegasian ontologi data ke Prolog dan eksekusi komputasi metrik oleh Python membuktikan bahwa arsitektur multi-domain ini mampu membantu insinyur menimbang berbagai kompromi (*trade-offs*) sistem rekayasa secara objektif.

Daftar Pustaka